Informatik von Grund auf — Band 17

Das Werk vieler Hände

Softwaretechnik



Vom Entwurf bis zur Ablösung

Jeder Schritt erklärt. Mit vollständig durchgerechneten Übungsaufgaben.

Stand der Forschung: August 2026

Informatik von Grund auf — Band 17
Das Werk vieler Hände · Softwaretechnik

Herausgeber und Urheber: Jürgen Maier
Entstanden 2026 in Zusammenarbeit mit Claude (Anthropic); Konzeption, Vorgaben, Prüfung und Herausgeberschaft: Jürgen Maier.

Lizenz
Dieses Werk ist lizenziert unter einer Creative-Commons-Lizenz **Namensnennung – Weitergabe unter gleichen Bedingungen 4.0 International (CC BY-SA 4.0)**. Sie dürfen das Werk vervielfältigen, verbreiten, bearbeiten und auch kommerziell nutzen, sofern Sie den Urheber nennen und Bearbeitungen unter derselben Lizenz weitergeben.
Lizenztext: `https://creativecommons.org/licenses/by-sa/4.0/legalcode.de`

Quellen und Fassungen
Quelltexte, Fehlermeldungen und aktuelle Fassungen aller Bände:
`https://github.com/juergenmaier2026/informatik-von-grund-auf`
Schwesterreihe: *Mathematik von Grund auf*, `https://doi.org/10.5281/zenodo.23698547`

Zitiervorschlag
Maier, J. (2026). *Informatik von Grund auf, Band 17: Das Werk vieler Hände*. Fassung 1.0.0.
`https://github.com/juergenmaier2026/informatik-von-grund-auf`

Satz: LaTeX mit Palatino. Fassung 1.0.0, Oktober 2026. 150 Seiten.

Inhaltsverzeichnis

Vorwort 1

I Der Gegenstand 9

1 Was Software anders macht 11

2 Größe und ihre Folgen 17

3 Der Lebenszyklus und seine Kosten 25

II Entwurf 33

4 Module 35

5 Architektur 43

6 Schnittstellen und Verträge 51

III Sicherstellung 59

7 Testen 61

8 Statische Prüfung 69

9 Formale Verfahren 77

IV Der Betrieb der Entwicklung 85

10 Versionsverwaltung und Integration 87

11 Vorgehensmodelle 95

12 Schätzen und Messen 103

V Die Zeit **111**

13 Wartung und technische Schuld **113**

14 Ablösung und Migration **119**

15 Der Stand im Jahr 2026 **127**

VI Anhang **133**

A Formeln und Kenngrößen **135**

B Landkarte und Glossar **139**

C Zeittafel **141**

D Kommentierte Quellen **143**

Vorwort: Warum Programmieren nicht genügt

Dieser Band handelt nicht davon, wie man ein Programm schreibt. *Er handelt davon, wie ein Programm entsteht, das größer ist, als ein Mensch überblicken kann, und das länger lebt, als seine Verfasser daran arbeiten.*

Der Unterschied

	Programmieren	Softwaretechnik
Verfasser	**einer**	**viele**
Zeitraum	Tage bis Wochen	**Jahre bis Jahrzehnte**
Leser	der Verfasser	**Fremde**
Maßstab	funktioniert es?	**lässt es sich ändern?**
Engpass	Können	**Verständigung**

Die letzte Zeile ist die These dieses Bandes. Bei einem Programm, das eine Person in einer Woche schreibt, entscheidet ihr Können. Bei einem System, an dem zwanzig Personen zehn Jahre arbeiten, entscheidet, wie gut sie einander verstehen — und wie gut das System sich selbst erklärt.

Warum Größe nicht linear wirkt.

Personen	Wege	je Person	
2	1	0,5	
6	15	2,5	
12	**66**	5,5	
30	435	14,5	unhaltbar

Die Zahl der Verständigungswege wächst quadratisch, die Arbeitskraft linear. Das ist die Rechnung auf dem Titelbild, und sie erklärt mehr über gescheiterte Projekte als jede Technik.

Ein Fach mit schwacher Beweislage

Softwaretechnik ist die am wenigsten belegte Teildisziplin der Informatik.

Aussage	Belegt?	
Fehler früh zu finden ist billiger	gut	
Kleine Teams sind produktiver	gut	
Sprache X ist besser als Y	**kaum**	Studien widersprüchlich
Vorgehen A schlägt B	**kaum**	
Metrik M misst Produktivität	**nein**	

> *Der Grund ist methodisch. Man kann kein Projekt zweimal durchführen, einmal so und einmal anders. Vergleichsstudien haben kleine Stichproben, keine Verblindung und eine Streuung zwischen Personen, die größer ist als jeder gemessene Effekt.*

Dieser Band sagt deshalb bei jeder Empfehlung dazu, worauf sie beruht: auf Messungen, auf Erfahrung, oder auf einem Argument. Wer das nicht trennt, verkauft Geschmack als Wissenschaft.

Vier Einsichten, die alt sind und ignoriert werden

Einsicht	Seit	Wer
Mehr Leute machen ein spätes Projekt später	1975	**Brooks**
Systeme spiegeln die Organisation	1968	**Conway**
Verbirg die Entwurf-sentscheidung	1972	**Parnas**
Software altert durch Änderung	1980	**Lehman**

Alle vier sind über vierzig Jahre alt, alle vier sind gut begründet, und alle vier werden regelmäßig verletzt. Das Fach hat kein Erkenntnisproblem, sondern ein Umsetzungsproblem — und dieser Band behandelt beides.

Was dieses Buch will

Erstens: rechnen, wo gerechnet werden kann.

> *Verständigungswege, Fehlerkosten über die Phasen, Testabdeckung, Schätzabweichungen, die Kosten einer Ablösung. Vieles in diesem Fach ist Meinung; einiges ist rechenbar, und dieser Band rechnet es aus.*

Zweitens: die Grenzen benennen.
Band 2 und der Begleitband zeigen, dass sich das Verhalten eines Programms nicht vollständig prüfen lässt. Kapitel 7 bis 9 ziehen daraus die praktischen Folgerungen: Was Testen leisten kann, was Typen garantieren, wann formale Verfahren sich lohnen.
Drittens: die Zeit ernst nehmen.

Kostenanteil	Wert	
Erstellung	≈ 30 %	
Pflege und Änderung	≈ **70** %	über die Lebensdauer

Der größere Teil der Arbeit fällt an, nachdem das System läuft. Ein Buch über Softwaretechnik, das dem Entwurf zehn Kapitel und der Wartung eines widmet, verfehlt seinen Gegenstand — hier sind es drei.

Drei Warnungen

Erstens: Es gibt kein Allheilmittel.
Brooks argumentierte 1987, dass keine einzelne Neuerung die Produktivität um eine Größenordnung heben wird, weil die verbleibende Schwierigkeit im Gegenstand liegt und nicht in den Werkzeugen. Vierzig Jahre später steht das Argument.
Zweitens: Vorgehensmodelle sind schwächer als ihre Anhänger glauben.

> *Der Unterschied zwischen zwei Personen ist in Messungen größer als der Unterschied zwischen zwei Vorgehensmodellen. Kapitel 11 führt die Zahlen vor — und auch die Geschichte des Wasserfallmodells, das nie jemand so empfohlen hat, wie es zitiert wird.*

Drittens: Produktivität lässt sich nicht messen.
Jede vorgeschlagene Kennzahl misst etwas, das leicht zu zählen ist, und lädt dazu ein, genau das zu vermehren. Kapitel 12 zeigt, warum.

Zum Aufbau

- **Teil I** (Kap. 1–3): Der Gegenstand. Was Software anders macht, was Größe bewirkt, wie die Kosten über die Zeit verteilt sind.
- **Teil II** (Kap. 4–6): Entwurf. Module, Architektur, **Verträge**.
- **Teil III** (Kap. 7–9): Sicherstellung. Testen, statische Prüfung, formale Verfahren.
- **Teil IV** (Kap. 10–12): Der Betrieb der Entwicklung. Integration, Vorgehen, Schätzen und Messen.
- **Teil V** (Kap. 13–15): Die Zeit. Wartung, Ablösung, der Stand im Jahr 2026.

Eine Tagung, die ein Wort erfand

1968 lud der NATO-Wissenschaftsausschuss nach Garmisch. Der Anlass war eine Beobachtung, die alle Beteiligten teilten und niemand erklären konnte.

Die Beobachtung	Große Softwareprojekte kamen zu spät, kosteten zu viel und taten nicht das Erwartete
Der Befund	Es lag nicht an einzelnen schlechten Programmierern
Das Wort	**Software Engineering** — bewusst provozierend gewählt

Der Begriff war eine Zumutung mit Absicht. Man wollte behaupten, dass Softwareherstellung eine Ingenieurdisziplin sein sollte — und zugleich eingestehen, dass sie es nicht war. Die Provokation hat gewirkt; der Anspruch ist bis heute nicht eingelöst.

Was im Bericht steht.

Ungewöhnlich	Der Bericht dokumentiert die Diskussionen wörtlich, samt Widerspruch
Der Ton	Ratlos, offen, ohne Beschönigung
Die Themen	Modularisierung, Prüfung, Schätzung, Wartung, Dokumentation

Die dritte Zeile ist die ernüchterndste. Die Tagesordnung von 1968 liest sich wie das Inhaltsverzeichnis dieses Bandes. In achtundfünfzig Jahren sind Werkzeuge und Größenordnungen andere geworden; die Fragen sind dieselben geblieben.

Und ein Wort, das dort geprägt wurde, ist geblieben: die Softwarekrise. Es beschreibt keinen vorübergehenden Zustand, sondern den Normalzustand eines Fachs, dessen Gegenstand schneller wächst als die Mittel, ihn zu beherrschen.

Sechzig Jahre Softwaretechnik

1962	Mariner 1	Softwarefehler zerstört eine Raumsonde
1966	Dijkstra	strukturierte Programmierung
1968	**NATO Garmisch**	**der Begriff entsteht**
1968	Conway	Systeme spiegeln die Organisation
1968	Dijkstra	*Go To Statement Considered Harmful*
1969	Hoare	**axiomatische Grundlage**
1970	Royce	das Wasserfallbild — als Warnung gemeint
1972	**Parnas**	**Geheimnisprinzip der Module**
1975	**Brooks**	***The Mythical Man-Month***
1976	Dijkstra	*A Discipline of Programming*
1980	**Lehman**	**Gesetze der Softwareevolution**
1981	Boehm	COCOMO, Schätzung mit Modell
1981	Hoare	*The Emperor's Old Clothes*
1986	Meyer	**Design by Contract**
1987	**Brooks**	***No Silver Bullet***
1987	DeMarco, Lister	*Peopleware*: der Mensch als Faktor

1988	Boehm	Spiralmodell: Risiko als Steuergröße
1993	—	**Pentium-Divisionsfehler**
1994	**Gamma u. a.**	**Entwurfsmuster**
1996	Ariane 5	**Wiederverwendung ohne Prüfung**
1997	Meyer	*Object-Oriented Software Construction*
1999	Kernighan, Pike	*The Practice of Programming*
2001	**Agiles Manifest**	**vier Wertepaare**
2002	Beck	testgetriebene Entwicklung
2004	Feathers	**Arbeiten mit Altlast**
2010	Humble, Farley	**fortlaufende Auslieferung**
2011	Leveson	Sicherheit als Systemeigenschaft
2018	Forsgren u. a.	**erste breite Messung**
2018	Ousterhout	Entwurf gegen Komplexität
2021	—	Lieferkette als Angriffsfläche
2026	—	Werkzeuge schreiben mit

Zwei Zeilen verdienen eine Anmerkung.
1970, Royce. Sein Aufsatz enthält das berühmte Stufenbild — und der Text darunter erklärt, dass dieses Vorgehen *nicht funktioniert* und Rückkopplungen nötig sind. Das Bild wurde zitiert, der Text nicht. Kapitel 11 zeigt, wie aus einer Warnung ein Standard wurde.
1996, Ariane 5. Eine Trägerrakete zerbrach vierzig Sekunden nach dem Start, weil eine Trägheitsplattform aus der Vorgängerreihe unverändert übernommen worden war. Der Code war korrekt — für die Ariane 4. Kapitel 6 behandelt den Fall als Lehrstück über Annahmen, die niemand aufgeschrieben hatte.

Ein Programm zu schreiben ist eine Frage des Könnens.
Ein System über zwanzig Jahre am Leben zu halten
ist eine Frage der Verständigung —
mit Menschen, die man nie treffen wird.

— der Leitgedanke dieses Bandes.
Er erklärt, warum die wirksamsten Mittel dieses Fachs
keine Werkzeuge sind, sondern Vereinbarungen.

Part I

Der Gegenstand

1 Was Software anders macht

Vier Eigenschaften des Gegenstands

FRED BROOKS nannte 1987 vier Merkmale, die Software von anderen Ingenieurserzeugnissen unterscheiden.

Komplexität	Kein Teil gleicht dem anderen; es gibt keine Wiederholung wie im Maschinenbau
Konformität	Die Schnittstelle richtet sich nach willkürlichen Vorgaben der Umwelt
Wandelbarkeit	**Man ändert Software, weil man es kann**
Unsichtbarkeit	**Es gibt keine natürliche räumliche Darstellung**

> *Die erste Zeile ist der Kern. Eine Brücke besteht aus Tausenden gleicher Nieten; ein Programm aus Tausenden verschiedener Anweisungen. Wo sich in der Fertigung Teile wiederholen, wird in der Software genau diese Wiederholung durch eine Schleife oder ein Unterprogramm beseitigt — übrig bleibt nur das Verschiedene.*

Daraus folgt eine unangenehme Eigenschaft: Software wächst nicht in die Breite, sondern in die Tiefe. Doppelte Größe bedeutet nicht doppelt so viele gleichartige Teile, sondern doppelt so viele verschiedene.

Die vierte Zeile im Detail.

Gegenstand	Darstellung	Vollständig?
Gebäude	Grundriss, Schnitt	ja
Schaltung	Schaltplan	ja
Programm	**mehrere Sichten**	**keine genügt**

Aufrufgraph, Datenfluss, Klassenhierarchie, Zeitverhalten, Abhängigkeiten — jede Sicht zeigt einen Schnitt und keine das Ganze. Ein Bauingenieur kann auf einen Plan zeigen; ein Softwareentwickler muss beschreiben. Das ist der Grund, warum Verständigung in diesem Fach so teuer ist.

Wesentliche und zufällige Schwierigkeit

	Zufällig	**Wesentlich**
Herkunft	**unsere Werkzeuge**	**der Gegenstand**
Beispiel	Assembler statt Hochsprache	die Sachlogik selbst
Behebbar	ja	**nein**
Anteil heute	gering	überwiegend

Brooks' Argument in einem Satz: Die zufälligen Schwierigkeiten sind weitgehend beseitigt, also kann keine weitere Werkzeugverbesserung eine Größenordnung bringen. Wer heute in einer Hochsprache mit Bibliotheken arbeitet, verbringt seine Zeit nicht mehr mit Speicherverwaltung, sondern mit der Frage, was das System eigentlich tun soll.

Fortschritt	**Gewinn**	**Art**
Hochsprachen	**Faktor** 5	zufällig
Zeitteilung	Faktor 2	zufällig
Integrierte Umgebungen	Faktor 1,5	zufällig
Bibliotheken	**Faktor** 3	teils wesentlich
Objektorientierung	$< 1{,}5$	
Vorgehensmodelle	unbelegt	

Die großen Sprünge liegen hinter uns, und sie waren alle Beseitigungen zufälliger Schwierigkeit. Was bleibt, ist die Aufgabe selbst — und die wird nicht einfacher, weil man ein besseres Werkzeug hat.

Das Argument ist stark und wird gern missverstanden

Brooks sagt	Keine *einzelne* Neuerung bringt binnen zehn Jahren Faktor zehn
Brooks sagt nicht	Dass Fortschritt unmöglich ist
Auch nicht	Dass viele kleine Verbesserungen nichts ergeben

Die Behauptung ist eng gefasst und deshalb überprüfbar. Seit 1987 hat keine Einzelneuerung sie widerlegt — weder Objektorientierung noch agile Verfahren noch Wolkendienste. Ob die heutigen schreibenden Werkzeuge es tun, ist die offene Frage von Kapitel 15.

Aufgabe 1.1.

(a) Bestimmen Sie, wie die Zahl der Zustände mit der Größe wächst.
(b) Trennen Sie zufällige von wesentlicher Schwierigkeit an einem Fall.
(c) Untersuchen Sie, warum es keine Sicht auf das Ganze gibt.

Lösung Schritt für Schritt.

(a) Zustände und Größe.
Die Rechnung. Ein Programm mit n unabhängigen booleschen Variablen hat 2^n Zustände.

Variablen	**Zustände**	**Prüfzeit bei 1 μs**	
10	1024	1 ms	trivial
20	10^6	1 s	
40	10^{12}	$\approx$ 12 Tage	
60	10^{18}	$\approx$ 36 000 Jahre	
100	10^{30}	jenseits allem	

Sechzig boolesche Variablen — weniger als ein durchschnittliches Formular — machen eine vollständige Prüfung unmöglich. Das ist keine Frage besserer Rechner, sondern eine Größenordnung, die keine Maschine erreicht. Kapitel 7 zieht daraus die Folgerungen.

Wie Struktur den Zustandsraum bricht.

Aufbau	Zustände	
60 Variablen, verkoppelt	$2^{60} \approx 10^{18}$	unprüfbar
6 Module zu 10	$6 \cdot 2^{10} = 6144$	**prüfbar**

Vierzehn Größenordnungen Unterschied, allein durch Trennung. Das ist die gesamte Begründung für Modularität in einer Zeile — und sie ist mathematisch, nicht ästhetisch. Kapitel 4 führt sie aus.

Die Voraussetzung steht im Wort verkoppelt. *Sechs Module ergeben nur dann $6 \cdot 2^{10}$ statt 2^{60} Zustände, wenn sie voneinander unabhängig sind. Jede Abhängigkeit, die man übersieht, holt einen Teil der Multiplikation zurück.*

(b) Zufällige und wesentliche Schwierigkeit.
Der Fall. Ein System zur Abrechnung von Krankenhausleistungen.

Schwierigkeit	Art	Behebbar durch
Speicher-verwaltung	zufällig	Sprache
Datenbankzugriff	zufällig	Bibliothek
Nebenläufigkeit	teils	bessere Sprachmittel
Abrechnungsregeln	**wesentlich**	**nichts**
Regeländerungen	**wesentlich**	nichts
Widersprüche in den Vorgaben	wesentlich	nichts

Die drei unteren Zeilen sind keine technischen Probleme. Sie sind Abbilder einer Sachlogik, die von Menschen gemacht ist, sich jährlich ändert und in sich widersprüchlich sein darf. Kein Werkzeug hilft — man muss die Regeln verstehen und in eine widerspruchsfreie Form bringen.

Die Aufwandsverteilung.

Tätigkeit	1975	2026	
Werkzeugkampf	≈ 35 %	≈ 10 %	gesunken
Sachlogik verstehen	≈ 25 %	≈ 35 %	
Verständigung	≈ 20 %	≈ 35 %	gestiegen
Eigentliches Codieren	≈ 20 %	≈ 20 %	gleich

Die letzte Zeile überrascht: Der Anteil des Programmierens ist konstant geblieben. Was gesunken ist, war der Kampf mit den Werkzeugen; was gestiegen ist, die Verständigung — weil die Systeme größer und die Teams zahlreicher wurden.

(c) Warum es keine Sicht auf das Ganze gibt.
Die Sichten und was sie zeigen.

Sicht	Zeigt	Verbirgt
Aufrufgraph	wer ruft wen	wann und wie oft
Datenfluss	woher Werte kommen	Ablauf
Klassen-hierarchie	Verwandtschaft	Zusammenarbeit
Ablauf-verfolgung	einen Lauf	alle anderen
Abhängigkeiten	Kopplung	Verhalten

Jede Spalte rechts ist der Preis für die Spalte in der Mitte. Ein Bauplan zeigt ein Gebäude vollständig, weil ein Gebäude im Raum liegt und der Plan eine Abbildung desselben Raums ist. Ein Programm liegt in keinem Raum.

Der Versuch, die Sichten zu zählen.

Systemgröße	Module	Sinnvolle Sichten	
10^3 Zeilen	5	3	überschaubar
10^5 Zeilen	50	8	
10^7 Zeilen	2000	≈ 20	niemand kennt alle

Ab einer gewissen Größe kennt kein Beteiligter mehr das ganze System, sondern nur seinen Ausschnitt und die Schnittstellen dorthin. Das ist kein Versäumnis, sondern eine Notwendigkeit — und der Grund, warum die Schnittstellen wichtiger werden als die Module.

Die Unsichtbarkeit hat eine praktische Folge

Beim Bau	Der Fortschritt ist sichtbar; ein halbes Haus sieht aus wie ein halbes Haus
Bei Software	**Ein zu neunzig Prozent fertiges System sieht aus wie ein fertiges**
Die Folge	Fortschritt lässt sich nur behaupten, nicht zeigen

Daraus stammt die bekannteste Redensart des Fachs: Die ersten neunzig Prozent brauchen neunzig Prozent der Zeit, die letzten zehn Prozent noch einmal neunzig. Sie ist ein Scherz und eine genaue Beschreibung — weil niemand von außen sieht, welche zehn Prozent fehlen.

Das ist auch der Grund, warum lauffähige Zwischenstände so wirksam sind. Sie ersetzen eine Behauptung über den Fortschritt durch etwas, das man vorführen kann — Kapitel 10 behandelt das systematisch.

Brooks[6] unterschied wesentliche von zufälliger Schwierigkeit; sein früheres Buch[5] enthält die Beobachtungen, auf denen das Argument beruht; der NATO-Bericht[25] dokumentiert, wie das Fach diese Fragen zuerst formulierte.

2 Größe und ihre Folgen

Brooks' Gesetz

Die Aussage	Fügt man einem verspäteten Projekt Personen hinzu, wird es noch später
Der Grund 1	Neue müssen eingearbeitet werden — von den Erfahrenen
Der Grund 2	**Die Verständigungswege wachsen quadratisch**

$$K(n) = \frac{n(n-1)}{2}.$$

n	**Wege**	**Zuwachs**	**je Kopf**
3	3	—	1,0
5	10	+7	2,0
8	28	+18	3,5
15	105	+77	7,0
30	**435**	+330	**14,5**

Die letzte Spalte ist die entscheidende: Bei dreißig Personen muss jede einzelne im Mittel vierzehn Beziehungen pflegen. Da ein Arbeitstag begrenzt ist, geschieht das nicht — die Verständigung bricht ab, und Teile des Systems entwickeln sich aneinander vorbei.

Die Aufteilbarkeit der Arbeit.

Art	Beispiel	Skaliert?
Vollständig teilbar	Baumwolle pflücken	**linear**
Teilbar mit Absprache	Softwarebau	unterlinear
Unteilbar	**ein Kind austragen**	**gar nicht**

Brooks' bekanntestes Bild: Neun Frauen bekommen kein Kind in einem Monat. Softwarebau liegt zwischen den Zeilen — teilbar, aber mit Absprachekosten, die mit dem Quadrat der Personenzahl steigen.

Conways Gesetz

Die Aussage	Ein System bildet die Verständigungsstruktur der bauenden Organisation ab
Warum	Wo zwei Teile zusammenarbeiten müssen, müssen die Teams sich absprechen
Die Folge	**Wo keine Absprache stattfindet, entsteht eine Schnittstelle**

Das Gesetz beschreibt keine Neigung, sondern eine Notwendigkeit. Ein Modul mit einer verwickelten inneren Schnittstelle zwischen zwei Teilen kann nur entstehen, wenn die zuständigen Personen eng zusammenarbeiten. Fehlt die Zusammenarbeit, entsteht stattdessen eine schmale, vereinbarte Grenze.

Beispiele.

Organisation	Ergebnis	
Vier Teams, ein Übersetzer	**vier Durchgänge**	Conway 1968
Getrennte Standorte	scharfe Schnittstellen	
Ein Raum, alle	**verwobenes System**	gut oder schlecht
Team pro Fachbereich	Module je Fachbereich	meist sinnvoll

Die dritte Zeile ist zweischneidig. Enge Zusammenarbeit erzeugt Systeme, deren Teile einander gut kennen — das ist schnell und schwer wieder zu trennen. Die Bewertung hängt davon ab, ob man Änderbarkeit oder Geschwindigkeit vorzieht.

Das Gesetz umdrehen

Wenn die Organisation die Architektur bestimmt, kann man die Architektur über die Organisation steuern.

Das Vorgehen	Zuerst die gewünschte Schnittstellenstruktur festlegen
Dann	Die Teams entlang dieser Struktur zuschneiden
Die Wirkung	Die Architektur entsteht, weil die Absprachen so verlaufen

Das ist ein wirksames Mittel und hat eine unangenehme Kehrseite: Eine Umorganisation ändert die Architektur, ob man will oder nicht. Wer Teams neu zuschneidet, ohne den Zuschnitt der Software zu ändern, erhält ein System, dessen Grenzen nicht mehr zu den Zuständigkeiten passen.

Aufgabe 2.1.

(a) Rechnen Sie Brooks' Gesetz an einem Projekt durch.

(b) Bestimmen Sie die günstigste Teamgröße.
(c) Prüfen Sie Conways Gesetz an einer Architektur.

Lösung Schritt für Schritt.

(a) Brooks' Gesetz durchgerechnet.
Die Lage. Ein Projekt mit 6 Personen ist zur Hälfte fertig und liegt 3 Monate zurück. Rest: 36 Personenmonate.
Ohne Verstärkung.

$$T = \frac{36}{6} = \mathbf{6}\ \text{Monate}.$$

Mit 6 zusätzlichen Personen.

Posten	**Aufwand**	
Einarbeitung je Neuem	1 PM	vom Erfahrenen
6 Neue	**6** PM	Abzug
Neue in Monat 1–2	50 % Leistung	-6 PM
Mehrverständigung	≈ 8 PM	siehe unten

Der Verständigungsaufschlag. Bei n Personen sinkt die nutzbare Leistung je Kopf um etwa $c \cdot (n-1)$ mit $c \approx 0{,}03$:

n	**Leistung je Kopf**	**Gesamt**	
6	0,85	5,1 PM/Monat	
12	0,67	8,0 PM/Monat	
18	0,49	8,8 PM/Monat	
24	0,31	7,4 PM/Monat	sinkt
30	0,13	3,9 PM/Monat	Zusammenbruch

Die Rechnung für den Fall.

Personen	Restaufwand	Rate	Dauer
6	36 PM	5,1	**7,1** Mon.
12	36 + 12 PM	8,0	**6,0** Mon.
18	36 + 24 PM	8,8	6,8 Mon.
24	36 + 36 PM	7,4	9,7 Mon.

Verdopplung bringt gut einen Monat, Vervierfachung kostet zweieinhalb. Die Kurve hat ein Minimum und steigt danach — genau das besagt Brooks' Gesetz. Bemerkenswert ist, dass das Minimum überhaupt existiert: Etwas Verstärkung hilft, viel schadet.

Und die entscheidende Größe ist der Aufschlag c. Er hängt davon ab, wie gut das System zerlegt ist. Bei sauber getrennten Modulen müssen nicht alle mit allen reden, und c sinkt — was Kapitel 4 zur wirtschaftlichen Frage macht.

(b) Die günstigste Teamgröße.
Das Modell. Gesamtleistung $L(n) = n \cdot (1 - c(n-1))$ mit c als Kopplungsmaß.
Das Maximum.

$$\frac{dL}{dn} = 1 - c(2n-1) = 0 \quad \Longrightarrow \quad n_{\text{opt}} = \frac{1}{2}\left(\frac{1}{c} + 1\right).$$

Kopplung	c	n_{opt}	Leistung
Sehr lose	0,01	50,5	25,5
Lose	0,02	25,5	12,8
Mittel	0,03	**17,2**	8,6
Eng	0,06	8,8	4,4
Sehr eng	0,12	**4,7**	2,3

Die günstigste Teamgröße hängt nicht vom Projekt ab, sondern von der Zerlegbarkeit der Aufgabe. Bei enger Kopplung liegt sie bei fünf Personen; bei loser bei fünfzig. Wer ein Team vergrößern will, muss zuerst die Kopplung senken.

Die empirisch beobachtete Größe wirksamer Teams liegt bei fünf bis neun Personen — was einem c zwischen 0,06 und 0,11 entspricht. Das ist ein Hinweis darauf, wie eng gekoppelt Softwarearbeit tatsächlich ist.

Die Aufteilung in mehrere Teams.

Aufbau	Wege intern	Wege extern	Summe
1×30	435	0	435
5×6	75	**10**	**85**
10×3	30	45	75

(extern: ein Vertreter je Team spricht mit den anderen Vertretern)

Aus vierhundertfünfunddreißig Wegen werden fünfundachtzig, allein durch Gruppierung. Das ist derselbe Bruch des Zustandsraums wie in Kapitel 1 — und er setzt dieselbe Bedingung voraus: Die Teams müssen tatsächlich getrennte Teile bearbeiten.

(c) Conways Gesetz an einer Architektur.
Der Befund. Ein System mit drei Diensten, gebaut von drei Teams.

Grenze	Aufrufe/Tag	Teams	Passt?
A–B	10^6	**dasselbe**	ja
B–C	10^3	verschieden	ja
A–C	**10^7**	**verschieden**	**nein**

Die dritte Zeile ist der Konflikt. Zwei Teile, die zehn Millionen Mal täglich miteinander sprechen, gehören technisch zusammen — und werden von getrennten Teams gepflegt. Jede Änderung an der Schnittstelle erfordert Abstimmung zwischen Organisationseinheiten.

Die beiden Auswege.

Weg	Man ändert	Aufwand
Architektur anpassen	A und C zusammenlegen	Monate
Organisation anpassen	**ein Team für A und C**	Wochen

Der zweite Weg ist meist billiger und wird selten erwogen, weil Organisationsfragen und Architekturfragen in verschiedenen Zuständigkeiten liegen. Conways Gesetz besagt, dass sie dieselbe Frage sind.

Die Zerlegung nach Schichten ist meist die falsche

Zuschnitt	Team je	Änderung betrifft
Nach Schichten	**Technik**	alle Teams
Nach Fachbereich	**Aufgabe**	ein Team

Ein Team für die Oberfläche, eines für die Sachlogik, eines für die Datenhaltung — das klingt ordentlich und bedeutet, dass jede fachliche Änderung drei Teams beschäftigt. Der Zuschnitt nach Fachbereichen ist technisch unsauberer und organisatorisch weit wirksamer.

Die Regel lautet: Man schneide entlang der Änderungen, nicht entlang der Technik. Kapitel 4 zeigt, dass Parnas dieselbe Regel 1972 für Module aufgestellt hat — lange bevor sie für Teams entdeckt wurde.

Brooks[5] formulierte das Gesetz und die Beobachtungen dahinter; Conway[7] beschrieb den Zusammenhang von Organisation und Systemstruktur auf wenigen Seiten; DeMarco und Lister[8] untersuchten die Bedingungen, unter denen Menschen in Projekten tatsächlich arbeiten.

3 Der Lebenszyklus und seine Kosten

Wo die Arbeit tatsächlich anfällt

Phase	Anteil	
Anforderungen	≈ 6 %	
Entwurf	≈ 8 %	
Codieren	≈ 7 %	
Prüfen	≈ 9 %	
Pflege und Änderung	≈ **70** %	nach der Übergabe

Die Zahlen streuen zwischen Studien erheblich — der Wartungsanteil wird zwischen 50 und 80 Prozent angegeben. Was nicht streut, ist die Aussage: Der größere Teil fällt an, nachdem das System läuft. Jede Entwurfsentscheidung sollte daran gemessen werden.

Was Wartung tatsächlich ist.

Art	Anteil	
Fehlerbehebung	≈ 20 %	
Anpassung an Umgebung	≈ 20 %	neue Plattform, Recht
Neue Anforderungen	≈ **50** %	**Weiterentwicklung**
Vorbeugende Arbeit	≈ 10 %	

Die dritte Zeile berichtigt ein verbreitetes Bild. Wartung ist überwiegend nicht Fehlerbehebung, sondern Weiterentwicklung — das System wird gepflegt, weil es benutzt wird und deshalb neue Wünsche erzeugt. Ein System ohne Wartungsaufwand ist meist ein System, das niemand braucht.

Lehmans Gesetze

MEIR LEHMAN untersuchte ab 1968 die Entwicklung großer Systeme über viele Jahre.

Fortwährende Änderung	Ein benutztes System muss geändert werden, sonst wird es unbrauchbar
Wachsende Komplexität	**Jede Änderung erhöht die Komplexität, sofern nicht gegengearbeitet wird**
Nachlassendes Wachstum	Die Zuwachsrate an Funktion sinkt über die Jahre
Erhaltene Vertrautheit	Änderungen je Ausgabe bleiben etwa konstant

Die erste Zeile ist die überraschendste. Ein System veraltet nicht, weil sein Code verschleißt — Code verschleißt nicht. Es veraltet, weil sich die Welt um es herum ändert. Ein unverändertes Programm wird mit jedem Jahr falscher.

Die zweite Zeile enthält das Wort, auf das es ankommt: sofern nicht gegengearbeitet wird. *Der Komplexitätsanstieg ist keine Naturkonstante, sondern der Normalfall bei fehlender Gegenarbeit. Kapitel 13 behandelt, worin diese besteht.*

Die Kosten einer Änderung über die Zeit

Gefunden in	Relativkosten	Beispiel
Anforderungen	1	Gespräch
Entwurf	3–6	Skizze ändern
Codieren	10	
Prüfen	15–40	
Nach Auslieferung	**40–1000**	je nach Feld

Die Spanne in der letzten Zeile ist ehrlich gemeint. Bei einer Webanwendung kostet ein nachträglicher Fehler wenig; bei ausgelieferter Gerätesoftware sehr viel. Die oft zitierte Zahl von tausend gilt für eingebettete Systeme und wird gern unzulässig verallgemeinert.

Der Grundbefund ist gut belegt und wird selten bestritten: Früh gefundene Fehler sind billiger. Die genauen Faktoren dagegen stammen aus wenigen alten Studien und sollten nicht auf zwei Stellen zitiert werden.

Aufgabe 3.1.

(a) Rechnen Sie die Gesamtkosten über die Lebensdauer aus.
(b) Prüfen Sie Lehmans zweites Gesetz an Zahlen.
(c) Bestimmen Sie, wann sich eine Investition in Änderbarkeit lohnt.

Lösung Schritt für Schritt.

(a) Kosten über die Lebensdauer.
Das System. 200 000 Zeilen, Erstellung 50 Personenjahre, Lebensdauer 15 Jahre.

Posten	Aufwand	
Erstellung	50 PJ	einmalig
Pflege je Jahr	≈ 8 %	4 PJ
15 Jahre Pflege	**60 PJ**	
Gesamt	**110 PJ**	Erstellung: 45 %

Bei steigender Pflegerate. Lehmans zweites Gesetz besagt, dass die Rate ohne Gegenarbeit steigt.

Jahr	Rate	Aufwand	kumuliert
1–3	6 %	3 PJ/a	9
4–7	9 %	4,5	27
8–11	14 %	7	55
12–15	**22** %	11	**99**
Gesamt			**149 PJ**

Aus hundertzehn werden hundertneunundvierzig Personenjahre — ein Aufschlag von fünfunddreißig Prozent, allein durch steigende Komplexität. Und die letzten vier Jahre kosten mehr als die gesamte Erstellung.

Die Frage, die daraus folgt.

Strategie	Gesamt	
Ohne Gegenarbeit	149 PJ	
Mit 10 % Aufschlag auf jede Änderung	**118 PJ**	Rate bleibt bei 8 %

Zehn Prozent Aufschlag auf jede Änderung, um die Struktur zu erhalten, sparen einunddreißig Personenjahre. Das ist die wirtschaftliche Begründung für das, was Kapitel 13 als fortlaufende Umgestaltung behandelt — und sie ist keine Geschmacksfrage.

(b) Lehmans zweites Gesetz an Zahlen.
Ein Maß für Komplexität. Die Zahl der Abhängigkeiten zwischen Modulen.

Ausgabe	Module	Kanten	Kanten/Modul
1,0	40	95	2,4
2,0	58	180	3,1
3,0	71	290	4,1
4,0	84	430	5,1
5,0	92	**610**	**6,6**

Die Module wachsen um Faktor 2,3*, die Abhängigkeiten um Faktor* 6,4*. Genau das besagt Lehmans zweites Gesetz: Die Struktur verschlechtert sich schneller, als das System wächst — weil jede neue Anforderung quer zu den bestehenden Grenzen liegt und leichter eine Abkürzung nimmt als eine saubere Trennung.*

Die Folge für die Änderbarkeit.

Ausgabe	Kanten/Modul	Betroffene Module	Aufwand
1,0	2,4	≈ 3	1,0
3,0	4,1	≈ 6	2,0
5,0	6,6	$\approx$ **12**	**4,0**

Dieselbe Änderung kostet in der fünften Ausgabe das Vierfache. Das erklärt die steigende Pflegerate aus Teil (a) — und es erklärt, warum erfahrene Teams Änderungen an alten Systemen so hoch schätzen, wie es Außenstehenden übertrieben vorkommt.

(c) Wann sich Investition in Änderbarkeit lohnt.
Die Rechnung. Ein Aufschlag a auf die Erstellung senkt die Pflegerate von p_0 auf p_1.

$$\text{Lohnt sich, wenn} \quad a \cdot E < (p_0 - p_1) \cdot E \cdot T,$$

also wenn

$$T > \frac{a}{p_0 - p_1}.$$

Aufschlag	p_0	p_1	Lohnt ab
20 %	12 %	8 %	**5 Jahre**
20 %	12 %	10 %	10 Jahre
50 %	12 %	6 %	8,3 Jahre
10 %	12 %	9 %	**3,3 Jahre**
100 %	12 %	4 %	12,5 Jahre

Die vorletzte Zeile ist die praktisch wichtigste: Ein kleiner Aufschlag mit mäßiger Wirkung amortisiert sich in gut drei Jahren. Die letzte Zeile zeigt die Grenze — wer die Erstellung verdoppelt, um die Pflege zu dritteln, braucht ein System, das über zwölf Jahre lebt.

Wann es sich nicht lohnt.

Fall	Lebensdauer	Urteil
Wegwerfprototyp	Wochen	nein
Auswertung einmalig	Tage	nein
Marktversuch	$<$ 1 Jahr	meist nein
Fachanwendung	10–20 Jahre	**ja**
Gerätesoftware	$>$ 15 Jahre	**unbedingt**

Die Antwort hängt allein an der erwarteten Lebensdauer, und die wird systematisch unterschätzt. Der häufigste Fehler ist nicht, in ein kurzlebiges System zu viel zu investieren, sondern ein langlebiges für kurzlebig zu halten.

Der Prototyp, der in Betrieb geht

Die Absicht	Schnell etwas zeigen, dann sauber neu bauen
Die Wirklichkeit	Der Prototyp funktioniert und wird produktiv genommen
Die Folge	**Ein System ohne Struktur mit zwanzig Jahren Lebensdauer**

Das ist der häufigste Weg, auf dem schwer wartbare Systeme entstehen — und er beruht auf keinem Fehler, sondern auf einem Erfolg. Wer einen Prototyp baut, sollte deshalb entweder verhindern können, dass er in Betrieb geht, oder ihn so bauen, als würde er es.

Brooks empfahl, von vornherein einzuplanen, das erste System wegzuwerfen — man werde es ohnehin tun. Kapitel 11 zeigt, dass er diesen Rat später zurückgenommen hat: Man wirft es nicht weg, sondern baut es schrittweise um.

Lehman[21] formulierte die Gesetze der Softwareevolution; Boehm[3] lieferte die Kostenmodelle und die Zahlen zur Phasenverteilung; McConnell[23] fasst die empirischen Befunde für die Praxis zusammen.

Part II

Entwurf

4 Module

Parnas' Kriterium

1972 stellte DAVID PARNAS zwei Zerlegungen desselben Programms gegenüber und fragte, welche besser sei.

	Nach Ablauf	**Nach Geheimnis**
Grenzen entlang	**Verarbeitungs-schritten**	**Entwurfs-entscheidungen**
Wirkt zunächst	natürlich	künstlich
Änderung betrifft	**alle Module**	**ein Modul**

Die Zerlegung nach Ablaufschritten ist die naheliegende: erst einlesen, dann sortieren, dann ausgeben. Sie ist auch die schlechtere — weil eine Änderung am Speicherformat jeden Schritt berührt, der Daten anfasst.

Das Kriterium	Jedes Modul verbirgt eine Entwurfsentscheidung, die sich ändern könnte
Die Schnittstelle	Verrät so wenig wie möglich über das Innere
Der Prüfstein	**Welche Änderung betrifft wie viele Module?**

Der Prüfstein ist bemerkenswert: Man beurteilt eine Zerlegung nicht an ihrer Schönheit, sondern daran, wie sie sich unter erwarteten Änderungen verhält. Das setzt voraus, dass man die erwarteten Änderungen benennt — und genau das unterbleibt meistens.

Kopplung und Kohäsion

	Kopplung	Kohäsion
Misst	**zwischen Modulen**	**innerhalb eines Moduls**
Gut ist	niedrig	hoch
Bedeutet	wenige Abhängigkeiten	**alles gehört zusammen**

Die Stufen der Kopplung, von schlecht nach gut.

Art	Bewertung	
Über gemeinsamen Zustand	**sehr schlecht**	niemand sieht es
Über innere Struktur	schlecht	bricht bei Umbau
Über Steuerflaggen	mäßig	
Über Datenstrukturen	mäßig	
Über schmale Schnittstelle	**gut**	

Die erste Zeile ist die gefährlichste, weil sie unsichtbar ist. Zwei Module, die über eine gemeinsame Variable verbunden sind, sehen im Aufrufgraphen unabhängig aus. Erst wenn eine Änderung an einer Stelle an einer anderen einen Fehler auslöst, wird die Verbindung sichtbar — und dann sucht man lange.

Wie man Kopplung misst

Statisch	Zahl der Verweise auf andere Module, aus dem Quelltext
Über Änderungen	**Welche Dateien werden gemeinsam geändert?**
Instabilität	$I = \frac{\text{ausgehend}}{\text{ein-+ausgehend}}$

Die zweite Zeile ist das aussagekräftigste Maß und wird am seltensten benutzt. Sie liest die Kopplung aus der Versionsgeschichte statt aus dem Quelltext — und findet dabei genau die verborgenen Verbindungen, die keine statische Analyse zeigt.

Aufgabe 4.1.

(a) Vergleichen Sie zwei Zerlegungen an einer Änderung.
(b) Berechnen Sie Kopplungsmaße.
(c) Bestimmen Sie die Kopplung aus der Änderungsgeschichte.

Lösung Schritt für Schritt.

(a) Zwei Zerlegungen.
Die Aufgabe. Parnas' Beispiel: Ein Programm, das Wortlisten einliest, zyklisch verschiebt, sortiert und ausgibt.
Zerlegung A, nach Ablauf.

Modul	Aufgabe	Kennt
1	Einlesen	**Speicherformat**
2	Verschieben	**Speicherformat**
3	Sortieren	**Speicherformat**
4	Ausgeben	**Speicherformat**

Zerlegung B, nach Geheimnis.

Modul	Verbirgt	Schnittstelle
1	**Speicherformat**	`zeichen(i,j)`
2	Eingabeformat	`lies()`
3	Verschiebeindex	`verschiebung(k)`
4	Sortierverfahren	`ordne()`
5	Ausgabeformat	`gib()`

Die Probe: fünf Änderungen.

Änderung	A betrifft	B betrifft
Speicherformat	**4**	**1**
Eingabeformat	1	1
Sortierverfahren	1	1
Zeichensatz	**4**	**1**
Verschiebung im Speicher statt vorab	**3**	**1**
Summe	**13**	**5**

Zerlegung B kostet mehr Module und weniger Arbeit — Faktor 2,6 über fünf Änderungen. Der Gewinn wächst mit der Lebensdauer, weil er sich bei jeder Änderung wiederholt, während der Mehraufwand einmalig ist.

Bemerkenswert an Parnas' Aufsatz ist, dass er die naheliegende Zerlegung nicht als Anfängerfehler abtut. Sie ist die natürliche, sie entspricht der Reihenfolge des Denkens — und sie ist trotzdem die schlechtere.

(b) Kopplungsmaße.
Das System. Sechs Module mit gerichteten Abhängigkeiten.

Modul	ein	aus	I	Rolle
A	0	3	**1,00**	**Anwendung**
B	1	2	0,67	
C	2	2	0,50	
D	3	1	0,25	
E	4	0	**0,00**	**Grundlage**
F	2	1	0,33	

Die Instabilität I misst, wie leicht ein Modul geändert werden kann. Bei $I = 1$ hängt niemand davon ab — man kann frei ändern. Bei $I = 0$ hängen vier Module davon ab — jede Änderung ist teuer. Beides ist richtig, solange es zur Rolle passt.

Die Regel der abnehmenden Instabilität.

Kante	I Quelle	I Ziel	Zulässig?
$A \to D$	1,00	0,25	ja
$B \to E$	0,67	0,00	ja
E → A	0,00	1,00	**nein**

Abhängigkeiten sollen in Richtung sinkender Instabilität zeigen: Wandelbares hängt von Stabilem ab, nie umgekehrt. Die dritte Zeile wäre der Fall, in dem eine Grundlagenkomponente die Anwendung kennt — und damit jede Änderung an der Anwendung die Grundlage bricht.

(c) Kopplung aus der Änderungsgeschichte.
Die Daten. Aus der Versionsverwaltung: welche Dateien in derselben Übergabe geändert wurden.

Paar	gemeinsam	X allein	Kopplungsgrad
X–Y	47	52	**0,90**
X–Z	12	52	0,23
Y–Z	3	60	0,05
P–Q	**31**	34	**0,91**

Die erste und die letzte Zeile zeigen Paare, die fast immer gemeinsam geändert werden. Das ist ein starker Hinweis auf eine Kopplung, die im Quelltext nicht sichtbar sein muss — etwa eine stillschweigende Übereinkunft über ein Datenformat.

Der Abgleich mit der statischen Analyse.

Paar	Statisch	Aus Historie	Befund
X–*Y*	Verweis da	0,90	stimmig
X–*Z*	Verweis da	0,23	Verweis kaum genutzt
P–Q	**kein Verweis**	**0,91**	**verborgene Kopplung**

Die dritte Zeile ist der Fund, den keine statische Analyse liefert. Zwei Module ohne jeden Verweis aufeinander werden in neunzig Prozent der Fälle gemeinsam geändert — es gibt also eine Abhängigkeit, die niemand aufgeschrieben hat und die jeder kennt, der lange genug dabei ist.

Solche Kopplungen sind die teuersten, weil sie neue Mitarbeiter nicht erkennen können. Sie zu finden ist die einfachste wirksame Auswertung, die man an einem gewachsenen System vornehmen kann.

Zu kleine Module sind auch schlecht

Modulgröße	Innere Komplexität	Schnittstellen	Summe
Sehr groß	hoch	wenige	mittel
Mittel	mäßig	mäßig	**gering**
Sehr klein	gering	**viele**	hoch

> *Zerlegung hat ein Optimum, kein Maximum. Wer ein System in zweihundert Kleinstmodule zerlegt, hat die Komplexität nicht beseitigt, sondern in die Schnittstellen verschoben — und dort ist sie schwerer zu überblicken, weil sie zwischen den Dateien liegt statt in ihnen.*

Ousterhout formuliert es zugespitzt: Ein Modul soll eine tiefe Schnittstelle haben — wenig nach außen, viel dahinter. Eine flache Schnittstelle, die kaum mehr verbirgt, als sie offenlegt, kostet mehr, als sie einbringt.

Parnas[27] stellte 1972 das Kriterium auf; Ousterhout[26] formuliert es als Forderung nach tiefen Schnittstellen neu; Meyer[24] verbindet es mit dem Begriff des Vertrags aus Kapitel 6.

5 Architektur

Was Architektur ist

Die Definition	Die Entscheidungen, die später teuer zu ändern sind
Nicht	Ein Diagramm mit Kästchen
Der Prüfstein	**Was kostet es, diese Entscheidung zurückzunehmen?**

Diese Bestimmung ist nützlich, weil sie eine Prüfung erlaubt. Eine Entscheidung, die man in einer Woche zurücknehmen kann, ist keine Architekturentscheidung, gleichgültig wie grundsätzlich sie klingt. Eine, die drei Jahre kostet, ist eine — auch wenn sie beiläufig getroffen wurde.

Entscheidung	Rücknahmekosten	Architektur?
Programmiersprache	Jahre	ja
Datenmodell	**Jahre**	**ja**
Verteilung auf Dienste	Jahre	ja
Rahmenwerk	Monate	ja
Sortierverfahren	Stunden	nein
Namensschema	Tage	nein

Das Datenmodell ist die teuerste Entscheidung überhaupt und wird am frühesten getroffen — zu einem Zeitpunkt, an dem man am wenigsten weiß. Kapitel 14 zeigt, dass Ablösungen fast immer daran scheitern.

Schichten und ihre Kosten

	Gewinn	Preis
Trennung	**Austausch-barkeit**	Umwandlung an jeder Grenze
Abstraktion	Verständlichkeit	Verlust an Steuerung
Indirektion	Erweiterbarkeit	schwerer nachvollziehbar

Jede Schicht kostet, und der Preis wird selten beziffert. Eine zusätzliche Umwandlungsschicht zwischen Datenbank und Sachlogik verlangt, dass jedes Feld zweimal beschrieben wird — und jede Erweiterung an zwei Stellen erfolgt.

Die Grundmuster.

Muster	Gut für	Schlecht für
Schichten	**Austausch-barkeit**	Durchsatz
Rohre und Filter	Verarbeitungs-ketten	Zustand
Ereignisse	lose Kopplung	Nachvollzieh-barkeit
Dienste	**getrennte Teams**	Übergreifende Änderung
Monolith	**Einfachheit**	unabhängige Auslieferung

Die letzten beiden Zeilen sind die derzeit meistdiskutierte Wahl, und die Entscheidung fällt seltener nach technischen als nach organisatorischen Kriterien — Conways Gesetz aus Kapitel 2. Wer ein Team hat, braucht keine getrennten Dienste.

Verteilung ist teuer

	im Prozess	über das Netz
Aufrufdauer	≈ 1 ns	≈ **500 ¯s**
Fehlerfälle	Ausnahme	**Teilausfall, Zeitüberschreitung**
Änderung an Schnittstelle	Übersetzer prüft	**niemand prüft**
Nachvollziehbarkeit	Aufrufkeller	verteilte Verfolgung nötig

Ein Netzaufruf kostet fünfhunderttausendmal so viel wie ein Aufruf im Prozess — und die dritte Zeile ist noch teurer. Wer eine Schnittstelle über das Netz zieht, verliert die Prüfung durch den Übersetzer und muss Verträglichkeit von Hand sicherstellen (Band 9).

Die vernünftige Reihenfolge lautet deshalb: erst modularisieren, dann verteilen — und nur, wenn ein Grund vorliegt. Ein verteiltes System aus schlecht getrennten Teilen hat alle Nachteile beider Bauformen.

Aufgabe 5.1.

(a) Bewerten Sie eine Schichtung nach ihren Kosten.
(b) Rechnen Sie die Latenz einer Aufrufkette aus.
(c) Prüfen Sie, wann Entwurfsmuster helfen.

Lösung Schritt für Schritt.

(a) Eine Schichtung bewerten.
Der Fall. Ein neues Feld soll von der Datenbank bis zur Oberfläche durchgereicht werden.

Schicht	Änderung	
Datenbankschema	1	
Zugriffsschicht	1	
Umwandlungs-objekt	1	
Sachlogik	1	
Übertragungs-objekt	1	
Oberfläche	1	
Berührte Stellen	**6**	für ein Feld

Sechs Stellen für ein Textfeld, das nirgends verarbeitet wird. Das ist der Preis der Trennung, und er fällt bei jeder Erweiterung an. Ob er sich lohnt, hängt davon ab, wie oft man tatsächlich eine Schicht austauscht.

Die Gegenrechnung.

Ereignis	Häufigkeit	
Neues Feld	≈ 40×/Jahr	je 0,5 h Mehraufwand
Datenbank gewechselt	≈ 0,05×/Jahr	einmal in 20 Jahren
Kosten	20 h/Jahr	
Nutzen	**≈ 15 h/Jahr**	knapp negativ

Die vollständige Trennung von der Datenbank rechnet sich nur, wenn man sie tatsächlich wechselt — und das geschieht fast nie. Das ist ein unbequemer Befund und wird gern übergangen, weil die Schichtung als guter Stil gilt.

Der ehrliche Schluss lautet nicht „keine Schichten“, sondern: Eine Schicht braucht eine Begründung, die über Ordnungsliebe hinausgeht. Wo die Austauschbarkeit nie in Anspruch genommen wird, hat man Kosten ohne Gegenwert.

(b) Latenz einer Aufrufkette.
Der Fall. Eine Anfrage durchläuft fünf Dienste.

Dienst	Eigenzeit	Netz	Summe
A	5 ms	0,5	5,5
B	12 ms	0,5	12,5
C	3 ms	0,5	3,5
D	20 ms	0,5	20,5
E	4 ms	0,5	4,5
Gesamt	44 ms	2,5	**46,5 ms**

Die Verfügbarkeit. Jeder Dienst sei zu 99,9 % verfügbar.

$$V = 0{,}999^{5} = \mathbf{0{,}9950}.$$

Dienste	Verfügbarkeit	Ausfall/Jahr	
1	0,9990	8,8 h	
5	0,9950	43,8 h	
20	0,9802	173 h	7 Tage
50	**0,9512**	**428 h**	18 Tage

> *Die Verfügbarkeit einer Kette ist das Produkt — sie sinkt mit jedem Glied. Fünfzig Dienste, jeder mit drei Neunen, ergeben zusammen weniger als zwei. Wer verteilt, muss deshalb von Anfang an mit Teilausfällen rechnen, statt sie als Ausnahme zu behandeln.*

(c) Wann Entwurfsmuster helfen.

Zweck	Wirksam?	
Als Vokabular	**sehr**	**ein Wort statt zehn Sätze**
Als Lösungskatalog	teilweise	wenn das Problem passt
Als Bauvorschrift	**schädlich**	Muster suchen Probleme

Der größte Nutzen von Entwurfsmustern liegt in der Verständigung, nicht im Entwurf. Wer sagt „das ist ein Beobachter“, spart eine Erklärung von zehn Sätzen — und das ist bei einem Fach, dessen Engpass die Verständigung ist, viel wert.

Die Kosten eines unnötigen Musters.

Lösung	Klassen	Zeilen	Sprünge
Direkter Aufruf	1	12	1
Mit Fabrik	3	48	3
Mit Fabrik und Strategie	6	**110**	**5**

Fünf Sprünge, um zu sehen, was tatsächlich ausgeführt wird. Wenn es je nur eine Umsetzung gibt, ist das kein Entwurf, sondern eine Vorratshaltung für einen Fall, der nicht eintritt.

Die Regel	Anzahl	
Erste Umsetzung	1	direkt schreiben
Zweite	2	noch abwarten
Dritte	3	**jetzt abstrahieren**

Die Regel der drei Fälle. Sie ist keine Wissenschaft, sondern Erfahrung — und sie ist gut begründet: Aus einem Fall lässt sich die richtige Verallgemeinerung nicht ablesen, aus zweien selten, aus dreien meistens. Wer zu früh abstrahiert, abstrahiert falsch.

Architekturentscheidungen sollten aufgeschrieben werden

Was fehlt	Nicht die Entscheidung, sondern **ihr Grund**
Die Folge	Nachfolger halten sie für willkürlich und ändern sie
Oder schlimmer	Sie halten sie für heilig und ändern sie nie

Beide Fehler haben dieselbe Ursache: Der Grund ist nicht überliefert. Eine kurze Notiz — Lage, Entscheidung, Begründung, verworfene Alternativen — kostet eine halbe Stunde und ist das billigste Mittel gegen beide.

Es ist dasselbe Prinzip wie bei den kommentierten Quellen dieser Reihe: Die Angabe allein nützt wenig; erst die Begründung macht sie brauchbar.

Fowler[14] beschreibt die wiederkehrenden Bauformen betrieblicher Anwendungen; Gamma und Mitautoren[15] lieferten das Vokabular der Entwurfsmuster; Ousterhout[26] argumentiert gegen die Verwechslung von Zerlegung mit Zergliederung.

6 Schnittstellen und Verträge

Eine Signatur ist kein Versprechen

Was die Signatur sagt	`wurzel(x: real) -> real`
Was sie nicht sagt	Dass $x \geq 0$ sein muss
Auch nicht	Dass das Ergebnis nichtnegativ ist und sein Quadrat x ergibt

Zwischen dem, was ein Übersetzer prüft, und dem, was gilt, klafft eine Lücke. Diese Lücke wird üblicherweise mit Kommentaren, Handbüchern oder mündlicher Überlieferung gefüllt — also mit Mitteln, die niemand prüft und die veralten.

Der Vertrag. BERTRAND MEYER schlug 1986 vor, sie aufzuschreiben und prüfen zu lassen.

Teil	Wer garantiert	Bedeutung
Vorbedingung	**der Aufrufer**	was gelten muss
Nachbedingung	**der Aufgerufene**	was danach gilt
Invariante	die Klasse	was immer gilt

Die Zuordnung in der mittleren Spalte ist der eigentliche Gedanke. Ein Vertrag verteilt Verantwortung: Der Aufrufer schuldet die Vorbedingung, der Aufgerufene die Nachbedingung. Wer eine Vorbedingung verletzt, hat den Fehler — und das ist eindeutig entscheidbar.

Warum Verantwortungsverteilung mehr ist als Prüfung

	Verteidigende Prüfung	**Vertrag**
Prüft	**jeder überall**	**einer, festgelegt**
Bei Verletzung	unklarer Fehlerwert	**Abbruch mit Schuldzuweisung**
Kosten	mehrfach	einmal

Ohne Vertrag prüft jede Funktion ihre Eingaben, weil sie dem Aufrufer nicht traut — und gibt bei Verletzung einen Fehlerwert zurück, den der Aufrufer wiederum prüfen muss. Ein erheblicher Teil des Codes besteht dann aus Misstrauen.

Ariane 5

Am 4. Juni 1996 zerbrach eine Trägerrakete siebenunddreißig Sekunden nach dem Start.

1	Die Trägheitsplattform wurde unverändert aus der Ariane 4 übernommen.
2	Eine Umwandlung von Gleitkomma nach 16-bit-Ganzzahl lief über.
3	Der Wert war die waagerechte Geschwindigkeit — bei der Ariane 5 **fünfmal höher**.
4	Die Ausnahme führte zur Abschaltung beider Rechner.
5	Die Steuerung deutete Diagnosedaten als Flugdaten und riss die Rakete quer.

Frage	Antwort	
War der Code falsch?	**nein**	für Ariane 4 korrekt
Gab es eine Vorbedingung?	ja	**nur in einer Analyse**
Stand sie im Code?	**nein**	
Wurde sie geprüft?	nein	bewusst entfernt

Die vierte Zeile ist die bitterste: Die Prüfung war aus Rechenzeitgründen weggelassen worden, gestützt auf eine Analyse, die zeigte, dass der Überlauf bei der Ariane 4 physikalisch unmöglich war. Die Analyse war richtig — und die Annahme, unter der sie galt, stand nirgends im Code.

Meyer nannte den Fall das teuerste Beispiel für einen fehlenden Vertrag. Ein Modul wurde wiederverwendet, ohne dass die Bedingungen seiner Gültigkeit mitgeliefert wurden — weil sie nicht Teil der Schnittstelle waren.

Die Lehre wird meist falsch gezogen

Behauptete Ursache	Zutreffend?	
Programmierfehler	nein	Code war korrekt
Ada war schuld	nein	Ada meldete den Fehler
Zu wenig getestet	teils	**Flugbahn nie simuliert**
Annahme nicht dokumentiert	**ja**	
Wiederverwendung ohne Prüfung	**ja**	

Die zweite Zeile ist bemerkenswert: Die Sprache hat den Überlauf erkannt und gemeldet — sie tat genau, was sie sollte. Das Versagen lag darin, dass niemand festgelegt hatte, was bei dieser Meldung geschehen soll. Die Voreinstellung war Abschaltung, und die war für eine Trägheitsplattform im Flug die schlechteste Wahl.

Aufgabe 6.1.

(a) Schreiben Sie Verträge für ein Modul auf.
(b) Rechnen Sie die Kosten defensiver Prüfung aus.
(c) Untersuchen Sie, wann Verträge nicht helfen.

Lösung Schritt für Schritt.

(a) Verträge aufschreiben.
Das Modul. Ein beschränkter Puffer.

Operation	Vorbedingung	Nachbedingung
`ablegen(x)`	**nicht voll**	Anzahl $+1$; x enthalten
`holen()`	**nicht leer**	Anzahl -1
`anzahl()`	—	$0 \le n \le \text{kap}$

Die Invariante.

$$0 \le \text{anzahl} \le \text{kapazität}.$$

Die Probe an einem Ablauf.

Aufruf	vorher	Vorbed.	nachher
`ablegen(a)`	0	erfüllt	1
`ablegen(b)`	1	erfüllt	2
`ablegen(c)`	2	erfüllt	$3 = \text{kap}$
`ablegen(d)`	3	**verletzt**	Abbruch
`holen()`	3	erfüllt	2

Beim vierten Aufruf bricht das Programm ab und nennt die verletzte Vorbedingung. Der Fehler liegt beim Aufrufer, der nicht geprüft hat — und das ist wichtiger als der Abbruch selbst: Man weiß sofort, wo zu suchen ist.

Der Unterschied zur Ausnahme.

	Vorbedingung	Ausnahme
Bedeutet	**Programmierfehler**	**erwarteter Sonderfall**
Beispiel	Puffer voll	Datei nicht gefunden
Behandlung	keine — beheben	abfangen
Im Betrieb	darf nie auftreten	tritt auf

Diese Unterscheidung wird häufig verwischt, und dann wird jeder Fehler zur Ausnahme und jede Ausnahme irgendwo abgefangen. Ein Vertragsbruch ist keine Ausnahme, sondern der Nachweis eines Fehlers im Programm.

(b) Kosten defensiver Prüfung.
Die Lage. Eine Funktion wird von zwölf Stellen aufgerufen.

Ansatz	Prüfstellen	Zeilen	Laufzeit
Jeder prüft alles	$12 + 1$	≈ 78	$13\times$
Nur die Funktion	1	6	$1\times$
Vertrag, im Betrieb aus	1	**2**	**0**$\times$

Die letzte Zeile beschreibt die übliche Praxis: Verträge werden beim Prüfen eingeschaltet und im Betrieb abgeschaltet, weil Fehler dann bereits gefunden sein sollten. Das ist wirtschaftlich sinnvoll und bei sicherheitskritischen Systemen falsch — Ariane 5 ist genau dieser Fall.

Wann man sie eingeschaltet lässt.

System	Prüfung	Grund
Spiel	aus	Geschwindigkeit
Betriebliche Anwendung	Vorbedingungen an	günstig
Medizingerät	**alles an**	**Abbruch besser als falsch**
Flugsteuerung	alles an	aber: was tun bei Bruch?

Die letzte Zeile enthält die schwierige Frage. Bei einem Medizingerät ist Abschalten die sichere Antwort; bei einer fliegenden Rakete nicht. Wo es keinen sicheren Zustand gibt, muss der Vertragsbruch behandelt werden statt abzubrechen — und das ist ungleich schwerer.

(c) Wann Verträge nicht helfen.

Eigenschaft	Fassbar?	
Wertebereiche	**ja**	leicht
Strukturelle Bedingungen	ja	Liste sortiert
Zeitverhalten	schwer	
Speicherverbrauch	schwer	
Gültigkeit der Annahme selbst	**nein**	**Ariane 5**
Sinnhaftigkeit des Ergebnisses	nein	

Die vorletzte Zeile ist die entscheidende Einschränkung. Ein Vertrag hält fest, was gelten muss — er sagt nicht, ob die zugrunde liegende Annahme über die Welt noch stimmt. Bei Ariane 5 war die Vorbedingung korrekt formuliert; falsch war die Überzeugung, dass sie automatisch erfüllt sei.

Was ein Vertrag dennoch geleistet hätte.

Sichtbarkeit	Die Bedingung stünde im Code, nicht in einer Analyse
Beim Übernehmen	**Wer das Modul wiederverwendet, liest sie**
Bei Verletzung	Eine benannte Ursache statt eines Zahlenüberlaufs

Die mittlere Zeile ist der eigentliche Wert. Ein Vertrag ist zuerst ein Dokumentationsmittel und erst danach eine Prüfung — und zwar eines, das im Code steht, mit ihm gepflegt wird und deshalb nicht veraltet.

Die Sprache mit Verträgen hat sich nicht durchgesetzt

Idee	Sprache	Heute
Design by Contract	Eiffel	Nische
Zusicherungen	**überall**	Standard
Verträge in der Sprache	Ada 2012	sicherheitskritisch
Typen als Verträge	**moderne Typsysteme**	**verbreitet**

Meyer hat verloren und gewonnen zugleich. Eiffel ist eine Randerscheinung geblieben; die Ideen sind überall — als Zusicherungen, als Verträge in Ada, und vor allem als ausdrucksstarke Typen, die Vorbedingungen in die Signatur ziehen und vom Übersetzer prüfen lassen.

Das ist der Weg, den Kapitel 8 verfolgt: Was sich in den Typ heben lässt, muss nicht zur Laufzeit geprüft werden — es kann gar nicht erst falsch geschrieben werden.

Meyer[24] entwickelte Design by Contract und wandte es auf den Ariane-Fall an; Hoare[16] lieferte 1969 die logische Grundlage der Vor- und Nachbedingungen; Leveson[22] untersucht Unfälle solcher Art als Systemversagen statt als Programmierfehler.

Part III

Sicherstellung

7 Testen

Was ein Test zeigen kann

Ein Test zeigt	Dass das Programm in *diesem* Fall richtig arbeitet
Er zeigt nicht	Dass es in allen Fällen richtig arbeitet
Dijkstras Satz	**Testen zeigt die Anwesenheit von Fehlern, nie ihre Abwesenheit**

Der Satz ist richtig und wird oft als Entwertung des Testens missverstanden. Er bedeutet nur: Testen ist Stichprobenprüfung. Das ist in fast allen Ingenieurdisziplinen der Normalfall — kein Brückenbauer belastet jede Brücke bis zum Bruch.

Die Testarten.

Art	Umfang	Dauer	Anteil
Einheit	eine Funktion	ms	$\approx 70\,\%$
Zusammenspiel	mehrere Module	s	$\approx 20\,\%$
System	alles	min	$\approx 10\,\%$

Die Verteilung heißt Testpyramide und ist eine Kostenüberlegung: Schnelle Tests dürfen zahlreich sein, langsame nicht. Wer sie umkehrt, erhält eine Prüfung, die Stunden dauert und deshalb nicht mehr bei jeder Änderung läuft — womit sie ihren Zweck verliert.

Abdeckung und was sie nicht misst

Maß	Prüft	Übersieht
Anweisungen	jede Zeile lief	Verzweigungen
Zweige	jeder Ausgang	Bedingungsteile
Bedingungen	jeder Teilausdruck	Kombinationen
Pfade	**jeder Ablauf**	**exponentiell viele**

Alle vier messen, was ausgeführt wurde — keines misst, ob geprüft wurde, dass es richtig war. Ein Test ohne einzige Zusicherung erreicht volle Anweisungsabdeckung. Das klingt absurd und kommt vor.

Abdeckung	Gefundene Fehler	
50 %	Bezugswert	
75 %	+35 %	lohnt sich
90 %	+50 %	
95 %	+53 %	abflachend
100 %	+55 %	sehr teuer

Der Ertrag flacht deutlich ab. Die letzten fünf Prozent kosten oft so viel wie die ersten siebzig und finden wenig — weil dort die Fehlerbehandlungszweige liegen, die selten falsch sind, aber schwer auszulösen.

Mutationstest: die Prüfung der Prüfung

1	Verändere den Quelltext geringfügig — ein Vergleichszeichen, eine Konstante.
2	Lasse die Tests laufen.
3	Schlägt kein Test fehl, ist die Änderung **überlebt**.
4	Der Anteil getöteter Mutanten misst die **Güte der Tests**.

Das ist das einzige Maß, das die Testsuite selbst bewertet statt den Code. Es ist teuer — die Tests müssen für jede Mutation vollständig laufen — und aussagekräftiger als jede Abdeckungszahl.

Aufgabe 7.1.

(a) Rechnen Sie die nötige Testzahl für verschiedene Abdeckungsmaße.
(b) Bestimmen Sie die Wirtschaftlichkeit der Testautomatisierung.
(c) Führen Sie einen Mutationstest durch.

Lösung Schritt für Schritt.

(a) Testzahl und Abdeckungsmaß.
Die Funktion. Drei unabhängige Bedingungen, verschachtelt.

```
if a and b:
if c:
tue X
else:
tue Y
else:
tue Z
```

Maß	Tests nötig	Beispiel
Anweisungen	**3**	X, Y, Z je einmal
Zweige	3	
Bedingungen	4	jeder Teil $\top$ und $\bot$
MC/DC	**4**	**Luftfahrtnorm**
Alle Kombinationen	**8**	2^3

Bei n Bedingungen.

n	MC/DC	Alle Komb.	Verhältnis
3	4	8	2,0
5	6	32	5,3
10	11	1024	93
20	21	$\mathbf{10^6}$	$\mathbf{5 \cdot 10^4}$

MC/DC wächst linear, die vollständige Kombinatorik exponentiell. Deshalb schreibt die Luftfahrtnorm MC/DC vor und nicht Vollständigkeit — es ist der stärkste Anspruch, der noch bezahlbar bleibt.

(b) Wirtschaftlichkeit der Automatisierung.
Die Lage. Ein Testfall, der von Hand 8 Minuten braucht.

Posten	Aufwand	
Automatisierung	3 h	einmalig
Pflege je Jahr	1 h	oft vergessen
Lauf automatisiert	5 s	
Lauf von Hand	8 min	

Der Umschlagpunkt.

$$3\,\text{h} + n_{\text{Jahre}} \cdot 1\,\text{h} < k \cdot 8\,\text{min}.$$

Läufe/Jahr	Hand, 3 J.	Automat, 3 J.	Urteil
2	0,8 h	6 h	Hand
12	4,8 h	6 h	knapp
50	20 h	6 h	Automat
500	200 h	**6 h**	**eindeutig**

Ab etwa fünfzehn Läufen im Jahr lohnt sich die Automatisierung. Da ein Test in einer fortlaufenden Integration bei jeder Übergabe läuft — also mehrere hundert Mal — ist die Antwort in der Praxis fast immer eindeutig.

Die Pflegezeile ist der Posten, der Rechnungen kippt und in ihnen fehlt. Ein Test, der bei jeder Umgestaltung angepasst werden muss, kostet dauerhaft — und das ist ein Argument dafür, gegen Schnittstellen zu testen statt gegen die innere Struktur.

(c) Ein Mutationstest.
Der Code.

```
def rabatt(betrag, jahre):
if jahre >= 5:
return betrag * 0.9
return betrag
```

Die Tests.

Fall	Eingabe	Erwartet
T_1	$(100, 2)$	100
T_2	$(100, 7)$	90

Die Mutanten.

Mutation	T_1	T_2	Ergebnis
>= zu >	ok	ok	**überlebt**
>= zu <	fällt	—	getötet
0.9 zu 0.8	ok	fällt	getötet
5 zu 6	ok	ok	**überlebt**
5 zu 4	ok	ok	überlebt
Getötet			**2 von 5**

Vierzig Prozent — bei hundert Prozent Zweigabdeckung. Die überlebenden Mutanten liegen alle an derselben Stelle: an der Grenze. Kein Test prüft, was bei genau fünf Jahren geschieht.

Der ergänzte Test.

Neu	Eingabe	Erwartet	
T_3	$(100, 5)$	**90**	Grenze
T_4	$(100, 4)$	100	darunter

Mit T_1–T_4	Getötet	
Alle fünf Mutanten	**5** von 5	100 %

Zwei zusätzliche Tests an der Grenze heben die Mutationsgüte von vierzig auf hundert Prozent. Das ist die alte Regel, hier hergeleitet statt behauptet: Fehler sitzen an den Rändern, nicht in der Mitte.

Tests sind selbst Code

Eigenschaft	Gilt auch	
Muss gepflegt werden	ja	
Kann Fehler enthalten	ja	**grüne Tests, die nichts prüfen**
Wird gelesen	ja	oft öfter als der Code
Umfang	1–3× Produktivcode	

Ein Testbestand ist so groß wie das System selbst oder größer — und unterliegt denselben Gesetzen. Schlecht strukturierte Tests altern wie schlecht strukturierter Code und werden irgendwann abgeschaltet statt repariert. Das ist der häufigste Weg, auf dem eine Testsuite stirbt.

Beck[1] beschreibt das testgetriebene Vorgehen und seine Wirkung auf den Entwurf; Kernighan und Pike[20] behandeln Prüfung als Teil der täglichen Arbeit; McConnell[23] fasst die empirischen Befunde zu Abdeckung und Fehlerfindung zusammen.

8 Statische Prüfung

Prüfen, ohne auszuführen

	Test	Statische Prüfung
Betrachtet	**einen Ablauf**	**alle Abläufe**
Findet	tatsächliche Fehler	auch mögliche
Irrt in Richtung	übersieht	Fehlalarm
Braucht	Testfälle	nur den Quelltext

Die dritte Zeile ist die entscheidende Ergänzung. Ein Test übersieht Fehler und meldet nie einen falschen; eine statische Prüfung meldet falsche und übersieht weniger. Beide irren nur in eine Richtung, und die Richtungen sind entgegengesetzt — deshalb ergänzen sie einander.

Warum keines von beiden vollständig sein kann.

Satz von Rice	Jede nichttriviale Verhaltenseigenschaft ist unentscheidbar
Die Folge	Kein Werkzeug findet alle Fehler und nur echte
Die Wahl	**Man entscheidet, in welche Richtung man irrt**

Band 2 und der Begleitband führen den Beweis; hier steht die praktische Folgerung. Jedes Prüfwerkzeug ist eine bewusste Entscheidung zwischen Fehlalarmen und übersehenen Fehlern — und diese Entscheidung sollte man kennen, bevor man dem Werkzeug glaubt.

Typen als billigste Prüfung

Stufe	Verhindert	Beispiel
Ohne Typen	—	alles möglich
Grundtypen	Verwechslung	Zahl statt Text
Eigene Typen	**Bedeutungs-verwechslung**	Meter statt Fuß
Zustandstypen	**falsche Reihenfolge**	Lesen vor Öffnen
Abhängige Typen	Wertebereiche	Index im Bereich

Die dritte Zeile ist die wirksamste und billigste Maßnahme dieses Kapitels. Wer für Meter und Fuß verschiedene Typen einführt, kann sie nicht mehr verwechseln — und genau diese Verwechslung kostete 1999 eine Marssonde.

Die vierte Zeile hebt Vorbedingungen in den Typ. Eine Datei, die geöffnet werden muss, bevor man liest, lässt sich so beschreiben, dass ein Lesen vor dem Öffnen nicht übersetzbar ist. Was Kapitel 6 zur Laufzeit prüft, wird hier unmöglich zu schreiben.

Die Kostenkurve der Fehlererkennung

Gefunden durch	Verzögerung	Kosten	
Übersetzer	Sekunden	1	billigst
Prüfwerkzeug	Minuten	2	
Einheitentest	Minuten	3	
Systemtest	Stunden	10	
Durchsicht	Tage	8	
Betrieb	Monate	**100**+	

Die Kosten hängen fast vollständig an der Verzögerung. Ein Fehler, den man Sekunden nach dem Schreiben sieht, kostet nichts — der Zusammenhang ist noch im Kopf. Nach drei Wochen muss man ihn sich neu erarbeiten, und das ist der eigentliche Aufwand.

Daraus folgt die Regel: Jede Prüfung so früh wie möglich. Nicht weil frühe Fehler harmloser wären, sondern weil das Wiedereinarbeiten teuer ist.

Aufgabe 8.1.

(a) Vergleichen Sie Fehlalarme und übersehene Fehler.
(b) Heben Sie eine Vorbedingung in den Typ.
(c) Rechnen Sie den Nutzen eines Prüfwerkzeugs aus.

Lösung Schritt für Schritt.

(a) Fehlalarme und übersehene Fehler.
Die Lage. Ein Prüfwerkzeug meldet 200 Stellen in einem System mit 40 echten Fehlern.

Einstellung	Meldungen	davon echt	gefunden
Streng	200	32	80 %
Mittel	70	24	60 %
Locker	25	**18**	45 %

Die Trefferquote.

Einstellung	Genauigkeit	Sichtungszeit	je echtem Fehler
Streng	16 %	200 × 5 min	**31 min**
Mittel	34 %	70 × 5	15 min
Locker	**72** %	25 × 5	**7 min**

> *Die strenge Einstellung findet die meisten Fehler und kostet je Fund viermal so viel. Schlimmer noch ist die Wirkung auf die Menschen: Bei einer Genauigkeit von sechzehn Prozent lernt man, Meldungen zu ignorieren — und übersieht dann auch die echten.*

Die praktische Regel lautet deshalb: lieber wenige, verlässliche Meldungen. Ein Werkzeug, dessen Ausgaben man ernst nimmt, findet am Ende mehr als eines, das alles meldet und ignoriert wird.

(b) Eine Vorbedingung in den Typ heben.
Das Problem. Eine Verbindung darf erst nach dem Öffnen benutzt werden.

Ansatz	Prüfung	Fehler bemerkt
Kommentar	keine	nie
Vorbedingung	zur Laufzeit	beim Test
Zwei Typen	**Übersetzer**	**beim Schreiben**

Die Umsetzung.

Operation	Von	Nach
`oeffnen`	`Geschlossen`	`Offen`
`lesen`	`Offen`	`Offen`
`schliessen`	`Offen`	`Geschlossen`

> *`lesen` nimmt nur eine `Offen`-Verbindung entgegen. Wer eine geschlossene übergibt, erhält einen Übersetzungsfehler — der Aufruf ist nicht bloß falsch, er ist unschreibbar. Das ist der stärkste Grad der Sicherstellung, den es gibt.*

Die Bilanz.

Ansatz	Mehr-aufwand	Laufzeit-kosten	Sicherheit
Kommentar	0	0	keine
Vorbedingung	2 Zeilen	je Aufruf	beim Test
Zwei Typen	≈ 15 Zeilen	**0**	**vollständig**

Fünfzehn Zeilen einmalig gegen vollständige Sicherheit ohne Laufzeitkosten. Das ist der beste Tauschhandel dieses Kapitels — und er wird selten gemacht, weil er beim Entwurf Aufwand verlangt und der Nutzen erst später anfällt.

(c) Der Nutzen eines Prüfwerkzeugs.
Die Zahlen. System mit 200 000 Zeilen, 20 Entwickler.

Posten	Wert	
Einrichtung	40 h	einmalig
Altlastmeldungen	≈ 900	Problem
Sichtung Altlast	75 h	
Laufende Meldungen	≈ 8/Woche	
Sichtung laufend	0,7 h/Woche	

Die zweite Zeile ist die Hürde, an der die Einführung meist scheitert. Neunhundert Meldungen im Bestand wirken erdrückend, und niemand fängt an. Der Ausweg ist bekannt und wird selten gewählt: Man friert die Altlast ein und prüft nur neuen Code.

Die Rechnung über ein Jahr.

Posten	Stunden	
Einrichtung	40	
Laufende Sichtung	36	$52 \times 0{,}7$
Aufwand	**76**	
Gefundene Fehler	≈ 35	
Ersparnis je Fehler	6 h	wenn früh gefunden
Nutzen	**210**	Faktor 2,8

Faktor zweikommaacht — ein solider, kein spektakulärer Gewinn. Bemerkenswert ist, wie stark er von der Ersparnis je Fehler abhängt: Bei einem System, dessen Fehler im Betrieb teuer sind, steigt sie auf ein Vielfaches, bei einem Wegwerfprogramm auf null.

Die Grenze der statischen Prüfung

Fehlerart	Findbar?	
Typverwechslung	**ja**	
Nullverweis	ja	bei passendem Typsystem
Nebenläufigkeits-fehler	teils	
Ressourcenleck	teils	
Falsche Sachlogik	**nein**	**niemals**

Die letzte Zeile ist die grundsätzliche Grenze. Kein Werkzeug weiß, was das Programm tun soll. *Ein Übersetzer prüft, ob der Code in sich stimmig ist; ob er die richtige Aufgabe löst, kann nur beurteilen, wer die Aufgabe kennt.*

Das ist auch die Antwort auf die Frage, ob Typen Tests ersetzen. Sie ersetzen eine Klasse von Tests vollständig — und lassen genau die Klasse übrig, auf die es fachlich ankommt.

Hoare[16] legte die logische Grundlage für Aussagen über Programme; Dijkstra[9] zeigte am Beispiel des Sprungbefehls, wie Sprachmittel die Nachvollziehbarkeit bestimmen; Hoare[17] beschreibt in seiner Turing-Award-Rede, was geschieht, wenn Prüfungen aus Bequemlichkeit unterbleiben.

9 Formale Verfahren

Beweisen statt prüfen

	Testen	Beweisen
Aussage über	**einige Fälle**	**alle Fälle**
Aufwand	mittel	sehr hoch
Voraussetzung	—	**eine formale Angabe dessen, was gelten soll**

Die dritte Zeile ist die eigentliche Hürde. Bevor man beweisen kann, muss man hinschreiben, was zu beweisen ist — vollständig und widerspruchsfrei. Bei den meisten Systemen ist genau das der schwierigste Teil, und er fällt auch ohne Beweis an.

Hoares Kalkül.

$$\{P\}\ S\ \{Q\}$$

P	Vorbedingung: was vor der Ausführung gilt
S	Die Anweisung
Q	Nachbedingung: was danach gilt

Das ist derselbe Begriff wie in Kapitel 6, nun als Beweisregel statt als Laufzeitprüfung. Hoare zeigte 1969, dass sich für jede Anweisungsart eine Regel angeben lässt, mit der man aus Teilbeweisen einen Gesamtbeweis zusammensetzt.

Die Schleifeninvariante

Der einzige nichtmechanische Schritt eines Programmbeweises.

Was sie ist	Eine Aussage, die vor und nach jedem Durchlauf gilt
Was sie leistet	Sie ersetzt die unendlich vielen Durchläufe durch einen Beweis
Warum schwer	**Sie muss erfunden werden — kein Verfahren findet sie**

Die Invariante ist die eigentliche Einsicht in eine Schleife: Sie sagt, warum die Schleife tut, was sie tut. Wer sie hinschreiben kann, hat das Programm verstanden — und deshalb ist ihr Wert unabhängig davon, ob man den Beweis zu Ende führt.

Wann sich der Aufwand lohnt

Bereich	Lohnt?	Grund
Fahrerlose Bahn	**ja**	**Menschenleben**
Prozessorentwurf	ja	Rückruf unbezahlbar
Kryptographisches Protokoll	ja	subtile Fehler
Betriebssystemkern	ja	**seit 2009 bewiesen**
Betriebliche Anwendung	nein	Vorgaben ändern sich
Webseite	nein	

Die vorletzte Zeile enthält den entscheidenden Grund gegen formale Verfahren im Regelfall: Man beweist gegen eine Angabe dessen, was gelten soll — und wenn diese Angabe sich vierteljährlich ändert, ist der Beweis vor seiner Fertigstellung veraltet.

Fall	Aufwand	
Pariser Metro, Linie 14	≈ 110 000 Zeilen	seit 1998 fehlerfrei
Verifizierter Kern	≈ 10 000 Zeilen	20 **Personenjahre**
Verifizierter Übersetzer	≈ 100 000 Zeilen	

Die mittlere Zeile ist die berühmteste Zahl des Gebiets: zwanzig Personenjahre für zehntausend Zeilen — etwa das Zwanzigfache des üblichen Aufwands. Dafür gibt es einen maschinengeprüften Beweis der funktionalen Korrektheit.

Was ein Beweis nicht beweist

Aussage	Bewiesen?	
Code erfüllt die Angabe	**ja**	
Die Angabe ist richtig	**nein**	**Menschenwerk**
Der Übersetzer ist korrekt	nein	außer man beweist ihn
Die Hardware arbeitet richtig	nein	
Die Annahmen treffen zu	nein	**Ariane 5**

Ein Beweis verschiebt die Frage vom Code auf die Angabe. Das ist ein echter Gewinn — die Angabe ist kürzer und wird sorgfältiger gelesen — und es ist keine Beseitigung des Problems. Die zweite Zeile bleibt, und sie ist genau die Stelle, an der die meisten Systeme scheitern.

Es ist dieselbe Kette wie beim Vertrauen in Übersetzer im Begleitband: Um einer Ebene zu trauen, braucht man die darunter — und irgendwo bricht man ab und macht die Annahme explizit.

Aufgabe 9.1.

(a) Beweisen Sie eine Schleife mit Invariante.
(b) Rechnen Sie die Wirtschaftlichkeit eines Beweises aus.
(c) Untersuchen Sie leichtgewichtige Verfahren.

Lösung Schritt für Schritt.

(a) Ein Schleifenbeweis.
Das Programm. Summe der ersten n Zahlen.

```
s := 0
i := 0
while i < n:
i := i + 1
s := s + i
```

Die Invariante.

$$I: \quad s = \frac{i(i+1)}{2} \ \wedge\ 0 \le i \le n.$$

Der Beweis in drei Schritten.
Erstens: Vor der Schleife gilt I.

$$s = 0, \quad i = 0, \qquad \frac{0 \cdot 1}{2} = 0. \qquad \checkmark$$

Zweitens: I bleibt erhalten. Es gelte I und $i < n$. Nach den beiden Zuweisungen:

$$i' = i + 1, \qquad s' = s + i' = \frac{i(i+1)}{2} + (i+1).$$

$$s' = \frac{i(i+1) + 2(i+1)}{2} = \frac{(i+1)(i+2)}{2} = \frac{i'(i'+1)}{2}. \qquad \checkmark$$

Drittens: Nach der Schleife folgt das Ziel. Die Schleife endet mit $\neg(i < n)$, also $i \ge n$; mit I folgt $i = n$:

$$s = \frac{n(n+1)}{2}. \qquad \square$$

Die Terminierung.

Terminierungsmaß	$n - i$	
Ganzzahlig	ja	
Nach unten beschränkt	≥ 0	aus I
Nimmt je Durchlauf ab	um 1	terminiert

Korrektheit und Terminierung sind getrennte Beweise. Der erste zeigt: wenn *die Schleife endet, ist das Ergebnis richtig. Erst der zweite zeigt, dass sie endet. Ein Programm kann bewiesen korrekt sein und ewig laufen.*

Bemerkenswert ist der Aufwand: eine halbe Seite für fünf Zeilen Code. Hochgerechnet auf zehntausend Zeilen ergibt das die zwanzig Personenjahre aus dem Kasten oben — die Zahl ist keine Übertreibung.

(b) Wirtschaftlichkeit.

Vorgehen	**Aufwand**	**Restfehler**	**je 10^3 Zeilen**
Nur Tests	1,0	≈ 1	
Tests und Durchsicht	1,3	$\approx 0{,}4$	
Statisch dazu	1,4	$\approx 0{,}2$	
Bewiesen	**20**	$\approx$ **0,002**	

Die Schadensrechnung.

Schaden je Fehler	**Lohnt Beweis?**	
1000 Euro	nein	Faktor 10^{-3}
10^5 Euro	nein	
10^7 Euro	knapp	
10^8 Euro	**ja**	Rückruf, Rakete
Menschenleben	**ja**	nicht in Geld

Der Umschlagpunkt liegt bei etwa zehn Millionen Euro Schaden je Fehler. Darunter sind Tests und Durchsicht wirtschaftlicher — sie erreichen ein Fünftel der Fehlerrate zum Bruchteil des Aufwands. Das ist kein Argument gegen formale Verfahren, sondern eines für ihren gezielten Einsatz.

(c) Leichtgewichtige Verfahren.

Verfahren	Aufwand	Findet
Modellprüfung	mittel	**Nebenläufigkeitsfehler**
Zusicherungen	gering	Verletzte Annahmen
Eigenschaftstests	gering	**Gegenbeispiele automatisch**
Verfeinerungstypen	mittel	Wertebereichsfehler
Abstrakte Interpretation	gering	Überläufe, Nullverweise

Die dritte Zeile ist der wirksamste Kompromiss dieses Kapitels. Statt Einzelfälle zu prüfen, schreibt man eine Eigenschaft hin — etwa „Sortieren erhält die Multimenge“ — und ein Werkzeug erzeugt Hunderte Fälle und schrumpft ein gefundenes Gegenbeispiel auf die kleinste Form.

Ein Beispiel.

Ansatz	Fälle	Fand den Grenzfall
Handgeschriebene Tests	8	nein
Eigenschaftstest	**1000**	**ja, in 2 s**
Beweis	—	ja, in 3 Tagen

Die mittlere Zeile ist die praktische Empfehlung: fast die Sicherheit eines Beweises für fast die Kosten eines Tests. Die Grenze liegt dort, wo der Zustandsraum zu groß für Stichproben ist — dann hilft nur die erste oder die letzte Zeile.

Die Angabe ist der Engpass, nicht der Beweis

Anteil am Aufwand	Wert	
Angabe formulieren	≈ 45 %	
Beweis führen	≈ 35 %	teils automatisch
Angabe berichtigen	≈ 20 %	**sie war falsch**

Die letzte Zeile ist der stille Hauptnutzen. Beim Versuch, eine Angabe formal hinzuschreiben, findet man Widersprüche und Lücken — und zwar bevor irgendetwas gebaut wurde. Mehrere Berichte über formale Projekte nennen das als größten Gewinn, unabhängig vom Beweis selbst.

Daraus folgt eine billige Empfehlung für Systeme, die keinen Beweis rechtfertigen: Man schreibe die Angabe formal, ohne zu beweisen. Der Nutzen fällt zu einem erheblichen Teil schon dort an.

Dijkstra[10] entwickelte das Programmieren aus der Ableitung statt aus dem Ausprobieren; Hoare[17] beschreibt, warum die formale Sorgfalt in der Praxis so oft unterbleibt; Leveson[22] zeigt an Unfällen, wo die Angabe und nicht der Code versagte.

Part IV

Der Betrieb der Entwicklung

10 Versionsverwaltung und Integration

Warum Zusammenführen exponentiell teurer wird

Trennzeit	Änderungen	Konflikte	Aufwand
1 Tag	≈ 8	0,2	5 min
1 Woche	≈ 40	2,5	40 min
1 Monat	≈ 170	**28**	6 h
3 Monate	≈ 500	**190**	5 Tage

Der Aufwand wächst überlinear, weil Konflikte einander bedingen: Wer eine Funktion umbenannt hat und jemand anders sie erweitert, muss beide Änderungen von Hand verweben — und je länger die Trennung dauert, desto weniger erinnern sich beide, warum sie es taten.

Daraus folgt die Empfehlung, die aussieht wie eine Werkzeugfrage und keine ist: Man führe oft zusammen. Nicht weil das Werkzeug es leichter macht, sondern weil die Erinnerung an den Grund einer Änderung schnell verblasst.

Was Versionsverwaltung tatsächlich leistet.

Wiederherstellen	Ein alter Stand ist zurückholbar — die schwächste Leistung
Zusammenführen	Mehrere arbeiten gleichzeitig am selben Bestand
Begründen	**Warum wurde diese Zeile so geschrieben?**
Auswerten	**Kopplung, Wissensverteilung, Alterungsherde**

Die dritte Zeile wird unterschätzt. Eine Versionsgeschichte ist das einzige Dokument, das mit dem Code gepflegt wird und deshalb nicht veraltet — vorausgesetzt, die Übergabetexte sagen warum *statt* was.

Fortlaufende Integration

1	Jede Übergabe löst einen automatischen Bau aus.
2	Alle Tests laufen.
3	Schlägt etwas fehl, ist die Behebung vorrangig.
4	Der Hauptzweig bleibt jederzeit lauffähig.

Punkt 4 ist die eigentliche Aussage, und sie ist eine Vereinbarung, keine Technik. Ein Bauserver erzwingt nichts — er meldet nur. Ob ein fehlgeschlagener Bau tatsächlich alles andere unterbricht, ist eine Frage der Disziplin.

Die Rückmeldedauer.

Dauer	Verhalten	Wirkung
< 2 min	man wartet	sofortige Behebung
5–10 min	Kaffee	brauchbar
30 min	man beginnt Neues	Umschalt-kosten
> 2 h	man ignoriert	**nutzlos**

Die letzte Zeile ist der häufigste Zustand gewachsener Systeme: Eine Prüfstrecke, die zwei Stunden braucht, wird nicht mehr abgewartet — und damit hat sie ihre Wirkung verloren, obwohl sie technisch einwandfrei läuft.

Die Prüfstrecke stufen

Stufe	Dauer	Umfang	Wann
1	90 s	Übersetzen, Einheitentests	jede Übergabe
2	12 min	Zusammenspiel	jede Übergabe
3	2 h	System, Last	nächtlich

Die Stufung löst den Widerspruch zwischen Gründlichkeit und Rückmeldedauer. Neunzig Sekunden fangen die Mehrzahl der Fehler ab; die teuren Prüfungen laufen, wenn niemand wartet. Der Preis ist, dass ein Fehler der dritten Stufe erst am nächsten Morgen auffällt.

Aufgabe 10.1.

(a) Rechnen Sie die Kosten langer Zweige aus.
(b) Bestimmen Sie die Wirkung der Rückmeldedauer.
(c) Lesen Sie die Wissensverteilung aus der Versionsgeschichte.

Lösung Schritt für Schritt.

(a) Kosten langer Zweige.
Das Modell. Bei n Änderungen auf jeder Seite und einer Kollisionswahrscheinlichkeit p je Paar:

$$K \approx p \cdot n_A \cdot n_B.$$

Mit $p = 0{,}004$ und gleich vielen Änderungen.

Dauer	n	Konflikte	Behebung
1 Tag	8	0,26	6 min
1 Woche	40	6,4	2,6 h
2 Wochen	80	25,6	10,2 h
1 Monat	170	**116**	46 h

(bei 24 min je Konflikt)

Vervierfachung der Trennzeit ergibt das Sechzehnfache an Konflikten — quadratisch, weil jede Änderung auf der einen Seite mit jeder auf der anderen kollidieren kann. Ein Monat Trennung kostet mehr als eine Arbeitswoche Zusammenführung.

Der Vergleich zweier Vorgehensweisen.

Vorgehen	Zusammen-führungen	Gesamt-aufwand/Jahr
Täglich	250	**25 h**
Wöchentlich	50	130 h
Monatlich	12	552 h

Zweiundzwanzigmal mehr Aufwand bei monatlicher gegenüber täglicher Zusammenführung. Das ist die stärkste Zahl dieses Kapitels und der eigentliche Grund für kurzlebige Zweige — nicht Modeerscheinung, sondern eine quadratische Kurve.

(b) Wirkung der Rückmeldedauer.
Der Umschaltverlust. Wer während einer Wartezeit eine andere Aufgabe beginnt, verliert beim Zurückwechseln Zeit.

Wartezeit	Umschalten?	Verlust	Gesamt
90 s	nein	0	1,5 min
8 min	teils	5 min	13 min
25 min	ja	**20** min	45 min
2 h	ja	20 min	140 min

Bei zehn Übergaben täglich und zwanzig Entwicklern.

Prüfdauer	je Tag	je Jahr	
90 s	5 h	1150 h	
8 min	43 h	**9900 h**	6 Personen-jahre

Der Unterschied zwischen einer eineinhalbminütigen und einer achtminütigen Prüfung beträgt bei diesem Team sechs Personenjahre — weit mehr, als die Beschleunigung der Prüfstrecke kosten würde. Das ist die wirtschaftlichste Einzelmaßnahme dieses Kapitels.

(c) Wissensverteilung aus der Versionsgeschichte.
Die Auswertung. Wer hat welche Dateien wie oft geändert?

Bereich	Personen	Hauptanteil	Risiko
Oberfläche	7	32 %	gering
Sachlogik	5	41 %	gering
Abrechnung	2	78 %	mittel
Schnittstelle Alt	**1**	**100** %	**hoch**

Die letzte Zeile ist der Befund, den man sucht: ein Bereich, den seit Jahren genau eine Person anfasst. Scheidet sie aus, ist das Wissen fort. Das ist keine Frage von Vertrauen, sondern von Risiko — und aus der Versionsgeschichte in Minuten ablesbar.

Die Alterungsherde.

Datei	Änderungen	Größe	Bewertung
A	340	2100 Z.	**Hotspot**
B	12	1800 Z.	stabil, groß
C	290	180 Z.	klein, aktiv
D	8	95 Z.	unauffällig

Datei A ist groß und wird ständig geändert — dort lohnt jede Umgestaltung am meisten. Datei B ist ebenso groß und wird nie angefasst; sie zu verbessern brächte nichts. Die Verbindung beider Größen ist die brauchbarste Anleitung dafür, wo man mit dem Aufräumen beginnt.

Was die Versionsgeschichte nicht misst

Größe	Aussage-kräftig?	
Kopplung	ja	Kap. 4
Wissensverteilung	ja	
Alterungsherde	ja	
Produktivität	**nein**	**Kapitel 12**
Qualität einer Person	nein	

Dieselben Daten, die für die Strukturanalyse hervorragend taugen, sind für die Bewertung von Personen wertlos und schädlich. Wer Übergaben zählt, misst Übergabegewohnheiten. Sobald die Zahl bekannt wird, steigt sie — ohne dass sich etwas anderes ändert.

Humble und Farley[18] beschreiben die durchgängige Auslieferungsstrecke; Feathers[11] zeigt, wie man Prüfstrecken um gewachsene Systeme herum aufbaut; Kernighan und Pike[20] behandeln Werkzeuge als Teil des Handwerks.

11 Vorgehensmodelle

Das Wasserfallmodell hat niemand empfohlen

WINSTON ROYCE veröffentlichte 1970 den Aufsatz, aus dem das Stufenbild stammt.

Abbildung 2	Die berühmte Stufenfolge — Anforderungen, Entwurf, Code, Prüfung, Betrieb
Der Text darunter	„Dieses Vorgehen *lädt zum Scheitern ein*“
Sein Vorschlag	**Rückkopplungen, ein Prototyp, mehrfaches Durchlaufen**

Der Aufsatz enthält zehn Abbildungen. Die zweite zeigt, was nicht funktioniert; die weiteren zeigen die Verbesserungen. Zitiert wurde die zweite — ein Lehrstück darüber, wie ein Bild einen Text besiegt.

Wie es dazu kam, ist gut belegt: Beschaffungsvorschriften des amerikanischen Verteidigungsministeriums übernahmen das Stufenbild als verbindliches Vorgehen. Damit wurde eine Warnung zur Norm — und blieb es zwei Jahrzehnte.

Was am Stufenmodell tatsächlich falsch ist.

Die Annahme	Anforderungen sind zu Beginn vollständig bekannt
Die Wirklichkeit	**Sie werden erst im Umgang mit dem System klar**
Die Folge	Der teuerste Fehler wird zum frühesten Zeitpunkt gemacht

Das ist kein Argument gegen Planung, sondern gegen eine *Planung zu Beginn. Bei einer Brücke sind die Anforderungen bekannt: Sie soll ein Flussbett überspannen und Lasten tragen. Bei einem Verwaltungssystem weiß niemand vorher genau, was es leisten soll.*

Was die Messungen hergeben

Unterschied zwischen	Faktor	Beleglage
Zwei Personen	**10–20**	gut belegt
Zwei Teams	≈ 5	gut
Zwei Vorgehens-modellen	≈ **1,2**	schwach
Zwei Sprachen	≈ 1,3	umstritten

Die erste und die dritte Zeile zusammen ergeben den unbequemsten Befund des Fachs: Wer die Personen austauscht, gewinnt eine Größenordnung; wer das Vorgehen austauscht, zwanzig Prozent. Die Debatten des Fachs führen wir überwiegend über die dritte Zeile.

Vorsicht ist bei der ersten Zeile angebracht: Der oft zitierte Faktor zehn stammt aus Studien der 1960er Jahre mit kleinen Stichproben und ist methodisch angreifbar. Dass die Streuung zwischen Personen groß ist, bestreitet dennoch niemand.

Das agile Manifest.

Eher	als
Menschen und Zusammenspiel	Verfahren und Werkzeuge
Lauffähige Software	**umfassende Dokumentation**
Zusammenarbeit mit dem Kunden	Vertragsver-handlung
Reagieren auf Änderung	**Befolgen eines Plans**

Der oft überlesene Nachsatz lautet: Die Werte rechts haben ebenfalls Bedeutung. Das Manifest verwirft die rechte Spalte nicht, es gewichtet. In der Rezeption ist daraus vielfach eine Ablehnung von Dokumentation und Planung geworden — was dort nicht steht.

Vorgehensmodelle werden zu Vorschriften

Gedacht als	Wird zu	
Wasserfall als Warnung	**Beschaffungsnorm**	1970er
Agil als Haltung	**Zertifikatswesen**	2010er
Besprechung zur Abstimmung	Statusbericht im Stehen	

Dasselbe Muster zweimal in fünfzig Jahren: Eine Empfehlung wird zur Vorschrift, die Vorschrift zum Prüfpunkt, und der Prüfpunkt ersetzt das Ziel. Wer eine tägliche Abstimmung in einen Statusbericht verwandelt, hat die Form behalten und den Zweck verloren.

Aufgabe 11.1.

(a) Vergleichen Sie die Kosten später Änderungen.

(b) Rechnen Sie den Wert früher Rückmeldung aus.

(c) Untersuchen Sie, was tatsächlich gemessen wurde.

Lösung Schritt für Schritt.

(a) Kosten später Änderungen.

Die Lage. Ein System, 24 Monate Laufzeit. Eine wesentliche Anforderung erweist sich als falsch.

Entdeckt in	Stufenmodell	Iterativ	Verhältnis
Monat 2	2 PM	2 PM	1,0
Monat 8	14 PM	5 PM	2,8
Monat 18	**62** PM	**11** PM	**5,6**
Monat 24	95 PM	14 PM	6,8

Der Unterschied entsteht nicht durch schnelleres Arbeiten, sondern durch den Zeitpunkt der Entdeckung. Im Stufenmodell wird die Anforderung erst bei der Abnahme geprüft; im iterativen Vorgehen nach jeder Ausbaustufe. Die Kosten sind dieselben — man findet den Fehler nur früher.

Die Wahrscheinlichkeit, dass es überhaupt vorkommt.

Anforderungen	Anteil geändert	
Bis Projektende	≈ 35 %	Median über Studien
Nie benutzt	≈ 45 %	einzelne Erhebungen

Die zweite Zeile ist mit Vorsicht zu genießen — sie stammt aus wenigen Erhebungen und wird gern übertrieben zitiert. Die Größenordnung ist trotzdem bemerkenswert: Ein erheblicher Teil der gebauten Funktion wird nie verwendet.

(b) Der Wert früher Rückmeldung.
Das Modell. Bei k Ausbaustufen wird ein Irrtum im Mittel nach $T/(2k)$ Zeit entdeckt.

Stufen	Entdeckt nach	Kosten je Irrtum	Bei 6 Irrtümern
1	12 Mon.	40 PM	240 PM
4	3 Mon.	15 PM	90 PM
12	1 Mon.	7 PM	**42** PM
24	0,5 Mon.	5 PM	30 PM

Der Preis der Stufung.

Stufen	Overhead	Irrtumskosten	Summe
1	0	240	240
4	8	90	98
12	24	42	**66**
24	48	30	78
52	104	24	128

Auch hier gibt es ein Optimum, kein Maximum. Zwölf Stufen bei zwei Jahren Laufzeit — also etwa monatlich — ist der günstigste Punkt; wöchentliche Auslieferung kostet durch den Overhead mehr, als sie an Irrtumskosten spart.

Der Overhead je Stufe ist dabei die Stellschraube. Wo Auslieferung automatisiert ist (Kapitel 10), sinkt er stark — und dann verschiebt sich das Optimum zu kürzeren Abständen. Das ist der sachliche Kern hinter der Empfehlung häufiger Auslieferung.

(c) Was tatsächlich gemessen wurde.
Der Befund aus großen Erhebungen.

Merkmal	Wirkung	Beleg
Häufige Auslieferung	stark	gut
Kurze Durchlaufzeit	stark	gut
Schnelle Wiederherstellung	stark	gut
Geringe Fehlerrate	stark	gut
Bestimmtes Rahmenwerk	keine	
Zertifizierte Rollen	keine	
Schätzverfahren	keine	

Die obere Hälfte misst Ergebnisse*, die untere* Verfahren*. Was sich als wirksam erwies, sind vier Ergebnisgrößen — und keine Vorschrift darüber, wie man sie erreicht. Das ist ein bemerkenswert bescheidenes Resultat für ein Fach mit so vielen Meinungen.*

Die methodische Einschränkung.

Problem	Gewicht	
Selbstauskunft	hoch	keine Messung am System
Selbstauswahl	hoch	wer antwortet?
Ursache oder Folge	**offen**	**beides plausibel**
Keine Verblindung	hoch	

Die dritte Zeile ist die entscheidende. Liefern gute Teams häufig aus, oder werden Teams gut, weil sie häufig ausliefern? Die Daten sind mit beidem verträglich. Das ist genau die Kausalitätsfrage aus Band 13, Kapitel 14 — und sie ist hier nicht entschieden.

Wer die Befunde dennoch nutzen will, findet ein gutes Argument in der Richtung der Wirkung: Häufige Auslieferung erzwingt kleine Änderungen, automatisierte Prüfung und schnelle Wiederherstellung. Es ist plausibel, dass die Praxis die Fähigkeit erzeugt — bewiesen ist es nicht.

Kein Modell ersetzt fähige Leute

Der Wunsch	Ein Verfahren, das auch mit durchschnittlichen Beteiligten gute Ergebnisse liefert
Der Befund	Der Personenunterschied überwiegt den Verfahrensunterschied um eine Größenordnung
Die Folge	**Einstellung und Einarbeitung sind wirksamer als Prozessarbeit**

Das ist unbequem, weil Prozessarbeit steuerbar ist und Personalauswahl nicht. DeMarco und Lister zogen daraus 1987 den Schluss, dass Arbeitsumgebung, Fluktuation und Störungsfreiheit stärker wirken als jedes Vorgehensmodell — und ihre Befunde sind bis heute nicht widerlegt.

Royce[28] entwarf das Stufenbild und warnte im selben Aufsatz davor; das agile Manifest[2] formulierte die Gegenposition in vier Wertepaaren; Boehm[4] schlug mit dem Spiralmodell einen risikogetriebenen Mittelweg vor.

12 Schätzen und Messen

Warum Schätzungen systematisch zu niedrig sind

Man schätzt	Den Weg, den man sich vorstellt
Man vergisst	**Alles, was man sich nicht vorstellt**
Die Verteilung	Rechtsschief — nach unten begrenzt, nach oben offen

Die dritte Zeile erklärt die systematische Unterschätzung ohne jede Annahme über Optimismus. Eine Aufgabe kann nicht weniger als null Zeit brauchen, aber beliebig viel mehr — also liegt der wahrscheinlichste Wert unter dem Erwartungswert. Wer den wahrscheinlichsten Verlauf schätzt, schätzt zu niedrig.

Größe	**Wert**	
Modalwert	5 Tage	**was man schätzt**
Median	6,5 Tage	
Erwartungswert	**8,4 Tage**	was es kostet
90-Perzentil	16 Tage	

Zwischen dem, was man nennt, und dem, was es im Mittel kostet, liegt bei dieser Verteilung Faktor 1,7 — ohne dass jemand geirrt hätte. Beide Zahlen sind richtige Antworten auf verschiedene Fragen.

Der Trugschluss der Planung.

Frage	**Antwort**	
Wie lange braucht es?	5 Tage	**eigene Schätzung**
Wie lange brauchte das letzte?	**9 Tage**	bessere Grundlage

Die zweite Zeile ist der wirksamste bekannte Ansatz: statt zu schätzen, vergleichbare abgeschlossene Vorgänge nachschlagen. Er ist zuverlässiger als jedes Schätzverfahren und wird selten benutzt, weil er unterstellt, dass man die Daten hat.

Metriken und ihre Verformung

Maß	Misst	Wird optimiert zu
Zeilen je Tag	Tippgeschwindigkeit	**längerer Code**
Behobene Fehler	Fehleranzahl	mehr Fehler einbauen
Übergaben	Gewohnheit	kleinere Übergaben
Testabdeckung	Ausführung	**Tests ohne Prüfung**
Erfüllte Schätzungen	—	höhere Schätzungen

Jedes dieser Maße ist für sich sinnvoll und wird schädlich, sobald es zur Zielgröße wird. Das ist keine Eigenheit der Softwaretechnik, sondern ein allgemeines Gesetz: Ein Maß, das zur Zielvorgabe wird, hört auf, ein gutes Maß zu sein.

Der schärfste Fall steht in der ersten Zeile. Wer Zeilen zählt, belohnt Umständlichkeit — und bestraft genau die Arbeit, die den größten Wert hat: das Entfernen von Code. Ein Tag, an dem ein System um vierhundert Zeilen schrumpft und dasselbe leistet, war ein guter Tag.

Was man messen kann, ohne Schaden anzurichten

Größe	Gefahr	
Durchlaufzeit einer Änderung	gering	schwer zu fälschen
Zeit bis zur Wiederherstellung	gering	
Anteil fehlgeschlagener Auslieferungen	mittel	
Zufriedenheit der Nutzer	gering	**das eigentliche Ziel**

Der gemeinsame Zug: Es sind Ergebnisgrößen des Gesamtsystems, nicht Leistungsgrößen einzelner Personen. Sie lassen sich kaum verbessern, ohne dass tatsächlich etwas besser wird — und genau das ist das Kriterium für ein brauchbares Maß.

Aufgabe 12.1.

(a) Rechnen Sie eine Schätzung mit Verteilung aus.
(b) Wenden Sie ein Aufwandsmodell an.
(c) Untersuchen Sie die Verformung durch eine Kennzahl.

Lösung Schritt für Schritt.

(a) Schätzung mit Verteilung.
Die Dreipunktschätzung. Optimistisch a, wahrscheinlich m, pessimistisch b:

$$E = \frac{a + 4m + b}{6}, \qquad \sigma = \frac{b - a}{6}.$$

Aufgabe	a	m	b	E	σ
1	2	4	12	5,0	1,67
2	1	2	5	2,3	0,67
3	5	8	20	9,5	2,50
4	3	5	14	6,2	1,83
5	1	3	9	3,7	1,33
Summe	12	22	60	**26,7**	

Die Gesamtstreuung. Bei unabhängigen Aufgaben addieren sich die Varianzen:

$$\sigma_{\text{ges}} = \sqrt{1{,}67^2 + 0{,}67^2 + 2{,}50^2 + 1{,}83^2 + 1{,}33^2} = \mathbf{3{,}70}.$$

Zusage	Tage	Treffer-wahrscheinlichkeit
Summe der Modalwerte	22,0	$\approx 11\,\%$
Erwartungswert	26,7	50 %
$E + \sigma$	30,4	84 %
$E + 2\sigma$	**34,1**	**98** %

> *Die erste Zeile ist die übliche Praxis — man addiert die wahrscheinlichen Werte — und sie trifft in einem von neun Fällen. Wer eine belastbare Zusage geben will, braucht den Erwartungswert plus eine Streuung, also fünfzig Prozent mehr als die naive Summe.*

Bemerkenswert ist der Ausgleichseffekt: Die Einzelstreuungen summieren sich zu 8,0, die Gesamtstreuung beträgt nur 3,7. Bei vielen unabhängigen Aufgaben gleichen sich Über- und Unterschreitungen teilweise aus — die Unsicherheit wächst mit der Wurzel, nicht linear.

(b) Ein Aufwandsmodell.
COCOMO in der Grundform.

$$\text{PM} = a \cdot (\text{KLOC})^b.$$

Art	a	b	Beispiel
Einfach	2,4	1,05	bekannte Aufgabe
Mittel	3,0	1,12	
Eingebettet	3,6	**1,20**	harte Randbedingungen

Für 50 **KLOC, mittlere Art.**

$$\text{PM} = 3{,}0 \cdot 50^{1{,}12} = 3{,}0 \cdot 78{,}4 = \textbf{235}\ \text{Personenmonate}.$$

Die Laufzeit.

$$T = 2{,}5 \cdot \text{PM}^{0{,}35} = 2{,}5 \cdot 235^{0{,}35} = \textbf{16{,}5}\ \text{Monate}.$$

$$\text{Mittlere Teamgröße} = \frac{235}{16{,}5} = \textbf{14{,}2}\ \text{Personen}.$$

KLOC	PM	Monate	Personen
10	39	9,2	4,3
50	235	16,5	14,2
200	1104	28,2	39
1000	**6488**	50,9	**127**

Der Exponent über eins ist die Aussage des Modells: Doppelte Größe kostet mehr als doppelten Aufwand. Von zehn auf tausend KLOC wächst der Umfang um Faktor hundert, der Aufwand um Faktor 166 *— das ist die quadratische Verständigung aus Kapitel 2 in einer Formel.*

Die unvermeidliche Warnung.

Datengrundlage	63 Projekte, überwiegend aus den 1970er Jahren
Eingabegröße	**Zeilenzahl — die man vorher nicht kennt**
Streuung	Faktor 2 nach oben und unten ist üblich

Die mittlere Zeile ist der grundsätzliche Einwand: Man braucht die Zeilenzahl, um den Aufwand zu schätzen, und kennt sie erst, wenn man fertig ist. Das Modell taugt für Größenordnungen und nicht für Zusagen.

(c) Verformung durch eine Kennzahl.
Der Versuch. Ein Team wird an behobenen Fehlern gemessen.

Quartal	Behoben	Gemeldet	Im Bestand
Vorher	85	90	210
1	112	105	203
2	138	**151**	216
3	164	**198**	**250**
4	181	224	**293**

Die gemessene Größe verdoppelt sich — und der Fehlerbestand steigt um vierzig Prozent. Was tatsächlich geschah: Kleinigkeiten wurden als Fehler gemeldet, um sie beheben zu können, und schwierige Fehler blieben liegen, weil sie zu lange dauerten.

Das bessere Maß.

Größe	Fälschbar?	
Behobene Fehler	leicht	
Fehlerbestand	mittel	
Von Nutzern gemeldete Fehler	**kaum**	**das eigentliche Ziel**

Die letzte Zeile misst, was zählt, und lässt sich vom Team nicht beeinflussen, außer durch bessere Arbeit. Das ist die Prüfregel für jede Kennzahl: Gibt es einen Weg, sie zu verbessern, ohne dass etwas besser wird?

Warum überhaupt geschätzt wird

Vorgeblich	Um die Dauer zu erfahren
Tatsächlich oft	Um eine Zusage zu erhalten, an der man festhalten kann
Die Folge	**Die Schätzung wird zum Versprechen**

Sobald eine Schätzung als Zusage behandelt wird, hört sie auf, eine Schätzung zu sein — sie wird zu einer Verhandlungsposition und entsprechend gepolstert. Danach messen beide Seiten etwas, das mit dem tatsächlichen Aufwand nichts mehr zu tun hat.

Der Ausweg ist nicht, nicht zu schätzen, sondern zu trennen: eine Schätzung mit Streuung für die Planung, eine Zusage mit Puffer für die Verpflichtung — und beide getrennt zu benennen.

Boehm[3] entwickelte die Aufwandsmodelle und ihre Datengrundlage; Jones[19] sammelte über Jahrzehnte Messwerte aus Projekten; Forsgren und Mitautoren[12] zeigen, welche Größen sich als aussagekräftig erwiesen haben.

Part V

Die Zeit

13 Wartung und technische Schuld

Das Bild von der Schuld

WARD CUNNINGHAM prägte 1992 einen Vergleich, der seither missbraucht wird.

	Finanzielle Schuld	Technische Schuld
Aufnahme	bewusst	**meist unbewusst**
Zinsen	bekannt	unbekannt
Tilgung	möglich	teilweise
Zweck	**schneller anfangen**	dasselbe

Cunninghams Gedanke war eng gefasst: Man liefert bewusst eine unvollständige Lösung aus, um früher zu lernen, und räumt danach auf. Was heute meist gemeint ist — angehäufte Nachlässigkeit — nannte er ausdrücklich nicht Schuld, sondern schlechte Arbeit.

Art	Absicht	Bewertung
Bewusst, umsichtig	**jetzt liefern, später räumen**	legitim
Bewusst, unbedacht	keine Zeit für Entwurf	riskant
Unbewusst, umsichtig	**man wusste es nicht besser**	unvermeidlich
Unbewusst, unbedacht	—	schlechte Arbeit

Die dritte Zeile ist die häufigste und wird selten genannt: Man hat nach bestem Wissen entworfen, und drei Jahre später hat man besseres Wissen. Das ist keine Schuld, sondern Lernen — und der Umgang damit ist derselbe.

Umgestaltung

Die Definition	Struktur ändern, ohne das Verhalten zu ändern
Die Voraussetzung	**Ein Prüfnetz, das Verhaltensänderungen meldet**
Ohne Prüfnetz	Kein Umgestalten, sondern Umschreiben auf Verdacht

Die zweite Zeile trennt das Verfahren von seinem Missbrauch. Wer ohne Tests umbaut, ändert das Verhalten und merkt es nicht — das ist keine Umgestaltung, sondern eine riskante Neufassung.

Wann sich Umgestaltung lohnt.

Änderungen pro Jahr	Aufwand jetzt	Ersparnis je Änderung	Amortisation
2	16 h	1,5 h	5,3 Jahre
12	16 h	1,5 h	0,9 Jahre
40	16 h	1,5 h	**0,27 Jahre**

Die Antwort hängt allein an der Änderungshäufigkeit — also an den Alterungsherden aus Kapitel 10. Wer dort aufräumt, wo selten geändert wird, verschwendet Zeit, gleichgültig wie schlecht der Code aussieht.

Die Pfadfinderregel

Die Regel	Hinterlasse den Code etwas besser, als du ihn vorgefunden hast
Der Vorzug	Aufräumen geschieht dort, wo ohnehin gearbeitet wird
Die Wirkung	**Selbstverstärkend — häufig geänderte Stellen werden am besten**

Die dritte Zeile ist der eigentliche Mechanismus. Die Regel führt automatisch zur richtigen Verteilung des Aufwands, ohne dass jemand eine Liste führen müsste — weil man nur dort vorbeikommt, wo Änderungen anfallen.

Aufgabe 13.1.

(a) Berechnen Sie die Zinsen technischer Schuld.
(b) Bringen Sie ein System ohne Tests unter Kontrolle.
(c) Bestimmen Sie, wo aufgeräumt werden soll.

Lösung Schritt für Schritt.

(a) Zinsen technischer Schuld.
Die Lage. Eine Abkürzung spart heute 3 Tage und kostet je künftiger Änderung 0,4 Tage zusätzlich.

Jahr	Änderungen	Zinsen	kumuliert
1	10	4,0 Tage	4,0
2	10	4,0	8,0
3	12	4,8	12,8
5	15	6,0	**24,0**

Drei gesparte Tage kosten nach einem Jahr vier und nach fünf Jahren vierundzwanzig. Der Zinssatz beträgt über hundert Prozent jährlich — kein Kreditgeber der Welt würde solche Bedingungen anbieten, und trotzdem wird diese Schuld täglich aufgenommen.

Wann sie sich trotzdem rechnet.

Fall	Urteil	
Marktfenster, 3 Wochen	**ja**	**Verzug kostet mehr**
Prototyp zur Prüfung	ja	wird verworfen
Dauerhaftes Kernmodul	**nein**	
„Wir räumen später auf“	nein	ohne festen Termin nie

Die erste Zeile ist Cunninghams ursprünglicher Fall und bleibt berechtigt: Wenn ein Verzug mehr kostet als die Zinsen, nimmt man die Schuld auf. Die letzte Zeile ist die häufigste Selbsttäuschung — eine Tilgung ohne Termin findet nicht statt.

(b) Ein System ohne Tests unter Kontrolle bringen.
Das Henne-Ei-Problem.

Um zu ändern	braucht man Tests
Um zu testen	muss man Abhängigkeiten auftrennen
Um aufzutrennen	**muss man ändern**

Der Ausweg in Schritten.

1	Finde die Stelle, an der geändert werden muss.
2	Bestimme die kleinste Naht — eine Stelle, an der sich Verhalten austauschen lässt.
3	Führe **minimale, risikoarme** Umbauten durch, um die Naht zu öffnen.
4	Schreibe einen Test, der das *jetzige* Verhalten festhält.
5	Ändere. Der Test meldet ungewollte Nebenwirkungen.

Schritt 4 ist der ungewöhnliche: Man prüft nicht, ob das Verhalten richtig ist, sondern hält fest, wie es ist *— auch wenn es falsch sein sollte. Ein solcher Test schützt vor unbeabsichtigten Änderungen, ohne dass man die Sachlogik verstanden haben muss.*

Die Wirkung über ein Jahr.

Monat	Abdeckung	Änderungs-dauer	Rückläufer
0	0 %	100 %	28 %
3	9 %	96 %	24 %
6	21 %	84 %	15 %
12	**38** %	**68** %	**7** %

Achtunddreißig Prozent Abdeckung nach einem Jahr — und die Rückläuferquote fällt von achtundzwanzig auf sieben Prozent. Die Abdeckung liegt dabei genau dort, wo gearbeitet wurde, also an den Alterungsherden. Eine gleichmäßige Abdeckung wäre teurer und weniger wirksam.

(c) Wo aufgeräumt werden soll.
Die Verbindung zweier Größen.

Datei	Komplexität	Änderungen	Rang
A	84	340	**28 560**
B	**210**	12	2520
C	31	290	8990
D	150	95	14 250
E	12	8	96

Datei B ist die komplexeste und steht auf Rang vier — sie wird kaum geändert, also lohnt die Arbeit nicht. Datei A ist nur mittelmäßig verwickelt und steht ganz oben, weil sie ständig angefasst wird. Das Produkt beider Größen ist die brauchbarste Reihenfolge.

Die Wirkung gezielten Aufräumens.

Vorgehen	Aufwand	Wirkung
Alles aufräumen	400 h	100 %
Nach Komplexität	120 h	≈ 35 %
Nach Rang	**120 h**	≈ **72** %

Dreißig Prozent des Aufwands für zweiundsiebzig Prozent der Wirkung — wenn man die richtigen Stellen wählt. Das ist die übliche Verteilung: Wenige Dateien tragen den größten Teil der Änderungskosten.

Die große Aufräumaktion scheitert fast immer

Der Plan	Drei Monate ohne neue Funktion, danach ist alles besser
Was geschieht	Nach sechs Wochen kommt eine dringende Anforderung
Das Ergebnis	**Halb umgebaut ist schlechter als gar nicht**

Ein halb vollzogener Umbau hinterlässt zwei Bauformen nebeneinander und Code, der beide kennen muss. Deshalb ist die schrittweise Umgestaltung entlang der laufenden Arbeit dem großen Wurf überlegen — nicht weil sie eleganter wäre, sondern weil sie jederzeit unterbrechbar ist.

Dasselbe Argument gilt eine Ebene höher für die vollständige Neuentwicklung — der Gegenstand von Kapitel 14.

Lehman[21] beschrieb das Wachsen der Komplexität als Gesetzmäßigkeit; Feathers[11] zeigt, wie man Systeme ohne Tests schrittweise unter Kontrolle bringt; Fowler[13] führt den Katalog der Umgestaltungsschritte aus.

14 Ablösung und Migration

Die vollständige Neuentwicklung

Die Verlockung	Diesmal machen wir es richtig, ohne die Altlast
Das Übersehene	**Die Altlast enthält zwanzig Jahre gelöste Sonderfälle**
Die Folge	Das neue System lernt sie noch einmal — einzeln

Das ist der Kern des Problems. Was im alten System unverständlich aussieht, ist häufig kein Pfusch, sondern die Antwort auf einen Sonderfall, den niemand mehr erinnert. Wer neu baut, entfernt die Antwort und behält das Problem.

Die Rechnung.

Posten	Aufwand	
Neuentwicklung, geschätzt	40 PJ	
Unbekannte Sonderfälle	+35 %	typisch
Parallelbetrieb	+8 PJ	
Datenüber-nahme	+6 PJ	unterschätzt
Weiterpflege des Alten	+12 PJ	**läuft weiter**
Tatsächlich	**80 PJ**	Faktor 2

Die vorletzte Zeile wird in Planungen fast immer vergessen: Das alte System muss während der gesamten Bauzeit weiterlaufen und weitergepflegt werden. Man baut nicht ein System, sondern zwei — und betreibt sie eine Zeitlang gleichzeitig.

Das zweite System

Brooks beschrieb 1975 einen wiederkehrenden Verlauf.

System	Merkmal	
Erstes	**schlank, vorsichtig**	man weiß wenig
Zweites	**überladen**	alles, was beim ersten fehlte
Drittes	wieder maßvoll	man hat gelernt

Beim zweiten System werden alle Wünsche eingebaut, für die beim ersten die Zeit fehlte — und es wird unter der Fülle unbrauchbar. Brooks nannte das die gefährlichste Stelle einer Laufbahn, weil sie mit Erfahrung und Selbstvertrauen zusammenfällt.

Die schrittweise Ablösung

1	Lege eine Fassade vor das alte System.
2	Leite alle Zugriffe darüber.
3	Ersetze **einen** Bereich hinter der Fassade durch das Neue.
4	Schalte um; bei Problemen zurück.
5	Wiederhole, bis nichts Altes mehr übrig ist.

Der entscheidende Vorzug steht in Schritt 4: Jeder Schritt ist einzeln umkehrbar. Bei der vollständigen Neuentwicklung gibt es genau einen Umschaltzeitpunkt, und wenn er misslingt, gibt es keinen Rückweg.

	Vollständig neu	Schrittweise
Risiko je Schritt	**alles**	gering
Nutzen ab	Ende	**erstem Schritt**
Umkehrbar	nein	ja
Gesamtdauer	kürzer	länger
Fassaden-aufwand	keiner	zusätzlich

Die letzten beiden Zeilen sind der ehrliche Preis. Das schrittweise Vorgehen dauert länger und verlangt eine Fassade, die man am Ende wieder entfernt. Man kauft Sicherheit mit Aufwand — und bei einem System, dessen Ausfall den Betrieb stilllegt, ist das ein guter Kauf.

Aufgabe 14.1.

(a) Vergleichen Sie die Risiken beider Wege.
(b) Rechnen Sie die Datenübernahme durch.
(c) Bestimmen Sie, wann eine Ablösung überhaupt lohnt.

Lösung Schritt für Schritt.

(a) Risiken beider Wege.
Die Erfolgswahrscheinlichkeit. Ein Umbauschritt gelinge mit $p = 0{,}92$.

Vorgehen	Schritte	Erfolg	
Vollständig neu	1 groß	$\approx 0{,}45$	**Schätzung**
Schrittweise, ohne Rückweg	12	$0{,}92^{12} = 0{,}37$	schlechter
Schrittweise, umkehrbar	12	$\approx$ **0,99**	**Wieder-holung**

Die mittlere Zeile ist der lehrreiche Fall: Zwölf Schritte ohne Rückweg sind schlechter *als ein großer, weil sich die Fehlerwahrscheinlichkeiten multiplizieren. Erst die Umkehrbarkeit dreht das Ergebnis — ein misslungener Schritt kostet dann Zeit statt das Vorhaben.*

Das ist eine allgemeine Einsicht über Risiko: Nicht die Zerlegung senkt es, sondern die Umkehrbarkeit der Teile. Wer zerlegt, ohne zurückgehen zu können, hat die Zahl der Gelegenheiten zu scheitern erhöht.

Der Schadensumfang.

Bei Fehlschlag	Verlust	
Vollständig neu	40–80 PJ	alles
Schrittweise	≈ 3 PJ	ein Schritt

Das Produkt aus Wahrscheinlichkeit und Schadenshöhe unterscheidet sich um mehr als zwei Größenordnungen. Bei der vollständigen Neuentwicklung wettet man das Vorhaben auf einen Wurf.

(b) Die Datenübernahme.
Der Bestand. 18 Jahre Betrieb, 12 Millionen Datensätze.

Befund	Anteil	
Sauber übertragbar	84 %	unproblematisch
Pflichtfeld leer	7 %	im Alten erlaubt
Widersprüchliche Verweise	5 %	
Historische Formate	3 %	**aus früheren Fassungen**
Unerklärbar	1 %	120 000 Sätze

> *Sechzehn Prozent der Daten passen nicht in das neue, sauberere Modell — und genau darin besteht die Arbeit. Das neue Modell ist strenger, weil man aus den Fehlern gelernt hat; die Altdaten entstanden unter den alten Regeln und verletzen die neuen.*

Die Entscheidungen, die niemand treffen will.

Möglichkeit	Preis	
Modell aufweichen	Altlast übernommen	
Daten bereinigen	≈ 4 PJ	**fachlich, nicht technisch**
Sätze verwerfen	Rechtsfragen	
Zwei Modelle führen	dauerhaft teuer	

Die zweite Zeile ist die richtige und die unbeliebteste: Datenbereinigung ist Facharbeit, nicht Programmierarbeit, und muss von denen geleistet werden, die den Sachverhalt kennen. Genau deshalb wird sie in Projektplänen als technische Aufgabe geführt und dort unterschätzt.

Der Aufwand im Vergleich.

Posten	Geplant	Tatsächlich
Übernahme-programme	1,5 PJ	2 PJ
Bereinigung	**0,5 PJ**	**4 PJ**
Abgleich und Prüfung	0,5 PJ	1,5 PJ
Summe	2,5	**7,5** Faktor 3

Der Faktor drei bei der Datenübernahme ist eine der verlässlichsten Erfahrungsregeln dieses Kapitels. Er entsteht fast vollständig in der zweiten Zeile — und die ließe sich vorab abschätzen, indem man die Altdaten gegen das neue Modell prüft, bevor man plant.

(c) Wann eine Ablösung lohnt.

Grund	Trägt?	
Plattform wird abgekündigt	**ja**	**Zwang**
Niemand beherrscht die Sprache	ja	Risiko
Änderungen dauern zu lange	prüfen	**ginge auch schrittweise**
Der Code ist unschön	**nein**	
Neue Technik ist verfügbar	**nein**	

Nur die beiden oberen Zeilen sind zwingende Gründe, und beide betreffen nicht die Software, sondern ihr Umfeld. Die dritte ist der häufigste genannte Grund — und meist mit gezielter Umgestaltung aus Kapitel 13 billiger zu lösen.

Die Gegenrechnung.

Weg	Aufwand	Ergebnis
Ablösung	80 PJ	modern, Risiko hoch
Gezielte Umgestaltung	**9 PJ**	Änderungsdauer −40 %
Nichts tun	0	Rate steigt weiter

Neun gegen achtzig Personenjahre — und die Umgestaltung wirkt ab dem ersten Monat, während die Ablösung erst nach Jahren etwas liefert. Wer die dritte Zeile der vorigen Tabelle als Grund nennt, sollte diese Rechnung aufmachen, bevor er sich für die erste Zeile entscheidet.

Ablösungen scheitern nicht an der Technik

Ursache	Anteil	
Unbekannte Fachlogik	**hoch**	**niemand kennt sie ganz**
Datenübernahme	hoch	
Bewegliches Ziel	hoch	**das Alte wird weitergebaut**
Technische Schwierigkeiten	gering	

Die dritte Zeile ist die tückischste: Während man das Neue baut, wird das Alte weiter geändert — und man verfolgt ein Ziel, das sich bewegt. Bei einer Bauzeit von vier Jahren und vierzig Änderungen im Jahr sind das hundertsechzig Änderungen, die nachzuziehen sind.

Daraus folgt die härteste Bedingung für jede Ablösung: Entweder das alte System wird eingefroren — was fachlich selten durchsetzbar ist — oder man löst schrittweise ab und pflegt nur noch einen Bestand.

Feathers[11] behandelt den Umgang mit gewachsenen Systemen; Fowler[13] beschreibt die schrittweise Ablösung hinter einer Fassade; sein Werk zu Anwendungsarchitekturen[14] liefert die Bauformen für die Zwischenschichten.

15 Der Stand im Jahr 2026

Was sich bewährt hat und was nicht

Ansatz	Beleglage	Urteil
Versionsverwaltung	eindeutig	unstrittig
Automatisierte Prüfung	gut	unstrittig
Frühe Rückmeldung	gut	unstrittig
Modularisierung	gut	unstrittig
Durchsicht durch Zweite	gut	wirksam
Statische Typprüfung	mittel	lohnt bei Größe
Testgetriebene Entwicklung	gemischt	personenabhängig
Paarprogrammierung	gemischt	situationsabhängig
Bestimmtes Rahmenwerk	keine	Geschmack
Zertifizierte Rollen	keine	Geschäftsmodell
Produktivitäts-metriken	negativ	**schädlich**

Die obere Gruppe ist bemerkenswert unspektakulär — es sind fast alles Maßnahmen, die Rückmeldung beschleunigen oder Umfang begrenzen. Vierzig Jahre Fachdiskussion haben keine einzige Erkenntnis hervorgebracht, die man 1975 nicht schon hätte formulieren können.

Fünf Entwicklungen

Erstens: Werkzeuge, die Code vorschlagen.

Wirkung auf	Befund	
Schreib-geschwindigkeit	deutlich	bei bekanntem Muster
Verstehen fremden Codes	merklich	
Entwurfsqualität	offen	
Verständigung im Team	**unver-ändert**	**der Engpass**
Prüfaufwand	steigt	mehr Code, mehr Durchsicht

Die vierte Zeile ist die Prüffrage an Brooks. Sein Argument lautete: Die zufällige Schwierigkeit ist weitgehend beseitigt, also kann kein Werkzeug mehr eine Größenordnung bringen. Wenn Schreiben schneller wird und Verständigung nicht, bleibt das Argument stehen — der Engpass hat sich nur nicht bewegt.

Die letzte Zeile enthält eine unbequeme Rückkopplung: Code, der schneller entsteht, muss ebenso sorgfältig gelesen werden — und Lesen ist nicht schneller geworden. Wer den Durchsatz vorn erhöht, verschiebt den Stau nach hinten.

Zweitens: Die Lieferkette als Angriffsfläche.

Größe	Wert	
Fremdanteil am Code	≈ 80–90 %	
Mittelbare Abhängigkeiten	**Hunderte**	unüberschaubar
Geprüft	fast nie	

Vier Fünftel eines heutigen Systems hat niemand im Team geschrieben oder gelesen. Das ist ein enormer Produktivitätsgewinn und zugleich Thompsons Vertrauensproblem aus dem Begleitband, nun in industriellem Maßstab.

Drittens: Sicherheit als Entwurfsfrage.

Fehlerklasse	Anteil	Vermeidbar durch
Speicherfehler	≈ 60 %	**Sprachwahl**
Eingabeprüfung	≈ 20 %	Typen
Rechtefehler	≈ 12 %	Entwurf

Die erste Zeile ist der stärkste bekannte Fall für eine Entwurfsentscheidung mit Sicherheitswirkung: Sprachen, die Speicherfehler ausschließen, beseitigen eine Fehlerklasse, die seit Jahrzehnten die Mehrheit der schweren Lücken stellt. Das ist Kapitel 8 in seiner folgenreichsten Anwendung.

Viertens: Systeme werden verteilt gebaut.

Merkmal	Wirkung	
Verteilte Teams	Conway wirkt stärker	Kapitel 2
Zeitzonen	Rückmeldung verzögert	
Schriftlichkeit	**Entscheidungen dokumentiert**	

Die dritte Zeile ist der unerwartete Gewinn: Wer nicht nebeneinander sitzt, muss schreiben — und schriftliche Entscheidungen sind das, was Kapitel 5 ohnehin fordert. Eine Beschränkung erzwingt eine gute Gewohnheit, wie in Band 16.

Fünftens: Die Systeme werden alt.

Bereich	Alter der Kernsysteme	
Banken	30–50 Jahre	COBOL
Versicherungen	25–40 Jahre	
Verwaltung	20–40 Jahre	
Industriesteuerung	15–30 Jahre	

Die tragenden Systeme der Wirtschaft sind älter als die meisten, die daran arbeiten. Damit wird Kapitel 13 und 14 zur Hauptbeschäftigung des Fachs — und die Lehre bildet weiterhin überwiegend für die Neuentwicklung aus.

Drei Einsichten

Erstens: Der Engpass ist die Verständigung, nicht das Schreiben.

Tätigkeit	Anteil	Seit 1975
Code schreiben	≈ 20 %	gleich
Code lesen	≈ 30 %	gestiegen
Verständigung	≈ 35 %	**gestiegen**
Werkzeuge	≈ 15 %	gesunken

Zwei Drittel der Arbeit bestehen aus Lesen und Reden. Deshalb wirken Maßnahmen, die Verständigung erleichtern — Modulgrenzen, Verträge, aufgeschriebene Entscheidungen — stärker als jede Beschleunigung des Schreibens.

Zweitens: Beschränkung erzwingt Entwurf.

Beschränkung	Erzwingt	
Kleines Team	**klare Grenzen**	Kap. 2
Schnelle Prüfstrecke	kleine Änderungen	Kap. 10
Enge Schnittstelle	Kapselung	Kap. 4
Verteilte Arbeit	**Schriftlichkeit**	

Dasselbe Muster wie in Band 16: Wer wenig hat, muss planen. Die praktische Folgerung ist ungewöhnlich — man führe Beschränkungen bewusst ein, wo sie technisch nicht mehr nötig sind, weil die Disziplin ohne Zwang nicht überlebt.

Drittens: Die Erkenntnisse sind alt, die Umsetzung fehlt.

Einsicht	Seit	Umgesetzt?
Geheimnisprinzip	1972	teilweise
Brooks' Gesetz	1975	**regelmäßig verletzt**
Conways Gesetz	1968	selten genutzt
Lehmans Gesetze	1980	selten beachtet
Kein Allheilmittel	1987	**jährlich vergessen**

Das Fach hat kein Erkenntnisproblem. Alles Wesentliche stand vor 1990 fest und ist gut begründet. Was fehlt, ist die Umsetzung — und das ist keine technische, sondern eine organisatorische Frage.

Der wiederkehrende Irrtum

Jahrzehnt	Das Versprechen	Ergebnis
1970er	Strukturierte Programmierung	nützlich
1980er	Werkzeug-umgebungen	mäßig
1990er	Objektorientierung	nützlich
2000er	Modellgetriebene Entwicklung	gering
2010er	Agile Rahmenwerke	gemischt
2020er	**schreibende Werkzeuge**	**offen**

Jedes Jahrzehnt bringt eine Neuerung, die als Lösung des Grundproblems angekündigt wird. Mehrere waren nützlich; keine hat den Faktor zehn gebracht. Brooks' Argument steht seit vierzig Jahren, und es steht nicht, weil die Neuerungen schlecht wären, sondern weil die verbleibende Schwierigkeit im Gegenstand liegt.

Die letzte Zeile ist offen, und dieser Band entscheidet sie nicht. Die Prüffrage lautet nicht, ob Code schneller entsteht — das ist bereits beantwortet — sondern ob Systeme dadurch verständlicher und änderbarer werden. Danach ist zu fragen, wenn in zehn Jahren Bilanz gezogen wird.

Alles Wesentliche dieses Fachs war 1990 bekannt.
Was seither hinzukam, waren bessere Werkzeuge
für Probleme, die nie die schwierigen waren.
Die schwierigen sind, was ein System tun soll
und wie man sich darüber verständigt.

— die Bilanz dieses Bandes.

Beides sind Fragen an Menschen, nicht an Maschinen —
und deshalb heißt der Band, wie er heißt.

Forsgren und Mitautoren[12] legten die breiteste Erhebung zu wirksamen Praktiken vor; Ousterhout[26] argumentiert, dass Komplexitätsbeherrschung die eigentliche Aufgabe bleibt; Brooks[6] formulierte 1987 die These, an der jede Neuerung seither zu messen ist.

Part VI

Anhang

A Formeln und Kenngrößen

Die Formeln

Verständigungswege	$K(n) = \frac{n(n-1)}{2}$
Günstigste Teamgröße	$n_{\text{opt}} = \frac{1}{2}\left(\frac{1}{c} + 1\right)$
Instabilität	$I = \frac{\text{ausgehend}}{\text{ein-} + \text{ausgehend}}$
Dreipunktschätzung	$E = \frac{a + 4m + b}{6}$
Streuung	$\sigma = \frac{b - a}{6}$
Gesamtstreuung	$\sigma_{\text{ges}} = \sqrt{\sum \sigma_i^2}$
Aufwandsmodell	$\text{PM} = a \cdot (\text{KLOC})^b$
Laufzeit	$T = 2{,}5 \cdot \text{PM}^{0{,}35}$
Verfügbarkeit einer Kette	$V = \prod_i v_i$
Zusammenführungskosten	$K \approx p \cdot n_A \cdot n_B$
Amortisation	$T > \frac{a}{p_0 - p_1}$

Die wichtigsten Zahlen

Größe	Wert	Kapitel
Anteil Pflege an Gesamtkosten	≈ 70 %	3
Davon Weiterentwicklung	≈ 50 %	3
Wirksame Teamgröße	5–9	2
Unterschied zwischen Personen	Faktor 10–20	11
Unterschied zwischen Vorgehen	Faktor ≈ 1,2	11
Geänderte Anforderungen	≈ 35 %	11
Fehlerkosten nach Auslieferung	40–1000×	3
Nutzen der Testautomatisierung ab	≈ 15 Läufe/Jahr	7
Erträgliche Prüfstreckendauer	< 10 min	10
Zusammenführung täglich statt monatlich	Faktor 22	10
Aufwand eines Beweises	≈ 20×	9
Ablösung gegen Schätzung	Faktor 2	14
Datenübernahme gegen Schätzung	Faktor 3	14
Fremdanteil am Code	80–90 %	15
Speicherfehler an schweren Lücken	≈ 60 %	15

Alle Zahlen sind Größenordnungen, keine Konstanten. Die Zeilen vier und fünf zusammen sind der wichtigste Befund der Tabelle — und der am wenigsten beachtete.

Die vier Gesetze

Brooks	Personen zu einem verspäteten Projekt hinzuzufügen verzögert es weiter
Conway	Ein System bildet die Verständigungsstruktur der Organisation ab
Parnas	Ein Modul verbirgt eine Entwurfsentscheidung, die sich ändern könnte
Lehman	Ein benutztes System muss sich ändern; dabei wächst die Komplexität

Vier Sätze, alle zwischen 1968 und 1980 formuliert, alle gut begründet, alle regelmäßig verletzt. Wer nur diese vier kennt und danach handelt, arbeitet besser als die meisten.

B Landkarte und Glossar

Wo was steht

Kap.	Thema	Verweise
1	Der Gegenstand	Band 2, Begleitband
2	Größe und Folgen	**Band 9**
3	Lebenszyklus	
4	Module	**Band 6**
5	Architektur	Band 9, Band 10
6	Verträge	**Band 0, Band 6**
7	Testen	**Band 2**
8	Statische Prüfung	**Band 2, Band 6**
9	Formale Verfahren	**Band 0, Band 2**
10	Integration	Band 11
11	Vorgehensmodelle	Band 13
12	Schätzen und Messen	**Band 13**
13	Wartung	Band 16
14	Ablösung	Band 10
15	Stand 2026	Begleitband

Begriffe

Abdeckung	Anteil des Codes, der beim Prüfen ausgeführt wurde
Architektur	Die Entscheidungen, die später teuer zu ändern sind
Fortlaufende Integration	Jede Übergabe wird automatisch gebaut und geprüft
Geheimnisprinzip	Ein Modul verbirgt eine Entwurfsentscheidung
Instabilität	Maß für die Änderbarkeit eines Moduls
Invariante	Aussage, die vor und nach jedem Durchlauf gilt
Kohäsion	Zusammengehörigkeit innerhalb eines Moduls
Kopplung	Abhängigkeit zwischen Modulen
MC/DC	Abdeckungsmaß der Luftfahrtnorm
Mutationstest	Prüfung der Testsuite durch gezielte Codeänderung
Nachbedingung	Was der Aufgerufene garantiert
Naht	Stelle, an der sich Verhalten austauschen lässt
Technische Schuld	Bewusst unvollständige Lösung mit späteren Kosten
Umgestaltung	Struktur ändern, ohne das Verhalten zu ändern
Vorbedingung	Was der Aufrufer schuldet

C Zeittafel

Von 1968 bis 2026

1968	**NATO-Tagung Garmisch: der Begriff entsteht**
1968	Conway: Systeme spiegeln die Organisation
1968	Dijkstra über den Sprungbefehl
1969	**Hoare: axiomatische Grundlage**
1970	Royce: das Stufenbild — als Warnung
1972	**Parnas: Geheimnisprinzip**
1975	**Brooks: The Mythical Man-Month**
1976	Dijkstra: A Discipline of Programming
1980	**Lehman: Gesetze der Softwareevolution**
1981	Boehm: COCOMO
1981	Hoare: The Emperor's Old Clothes
1986	Meyer: Design by Contract
1987	**Brooks: No Silver Bullet**
1987	DeMarco, Lister: Peopleware
1992	Cunningham: das Bild der Schuld
1994	**Gamma u. a.: Entwurfsmuster**
1996	Ariane 5
2001	**Agiles Manifest**
2002	Beck: testgetriebene Entwicklung
2004	Feathers: Arbeiten mit Altlast
2009	erster maschinell verifizierter Betriebssystemkern
2010	Humble, Farley: fortlaufende Auslieferung
2011	Leveson: Sicherheit als Systemeigenschaft
2018	**Forsgren u. a.: breite Messung**
2018	Ousterhout: Entwurf gegen Komplexität
2021	Lieferkette als Angriffsfläche
2026	Werkzeuge schreiben mit

Zwei Zeilen verdienen eine Anmerkung.
1970, Royce. Der Aufsatz, dessen zweite Abbildung als Vorschrift zitiert wurde, obwohl der Text sie verwirft. Ein Bild besiegte einen Text und prägte zwei Jahrzehnte Beschaffungspraxis.
1987, Brooks. Die einzige Behauptung dieses Fachs, die sich als Prognose prüfen lässt — und sie hat vierzig Jahre gehalten. Kapitel 15 stellt die Frage, ob das so bleibt.

D Kommentierte Quellen

[1] Beck, K.: *Test-Driven Development: By Example*. Addison-Wesley 2002.

Beschreibt das Verfahren an durchgerechneten Beispielen statt in Grundsätzen. Der eigentliche Gehalt liegt in der Wirkung auf den Entwurf — wer zuerst den Test schreibt, entwirft schmalere Schnittstellen. Die Wirksamkeitsstudien sind gemischt.

[2] Beck, K., u. a.: *Manifesto for Agile Software Development*. 2001.

Vier Wertepaare auf einer Seite. Lesenswert vor allem wegen des Nachsatzes, dass die Werte rechts ebenfalls Bedeutung haben — die Passage, die in der Rezeption regelmäßig fehlt.

[3] Boehm, B.: *Software Engineering Economics*. Prentice Hall 1981.

Der erste ernsthafte Versuch, das Fach wirtschaftlich zu fassen. *Enthält COCOMO und die Zahlen zur Phasenverteilung. Die Datengrundlage ist alt und die Denkweise trägt bis heute.*

[4] Boehm, B.: *A Spiral Model of Software Development and Enhancement*. IEEE Computer 21(5), 1988.

Stellt das Risiko in den Mittelpunkt statt der Phasen: In jeder Runde wird zuerst bearbeitet, was am meisten schiefgehen kann. Der klügste Kompromiss zwischen Stufenmodell und iterativem Vorgehen.

[5] Brooks, F.: *The Mythical Man-Month*. Addison-Wesley 1975, erweitert 1995.

Das wichtigste Buch dieses Fachs. *Enthält Brooks' Gesetz, das zweite System und die Einsicht, dass Verständigung der Engpass ist. Die Ausgabe von 1995 enthält Berichtigungen des Autors an sich selbst — darunter die Rücknahme des Rats, das erste System wegzuwerfen.*

[6] Brooks, F.: *No Silver Bullet: Essence and Accidents of Software Engineering*. IEEE Computer 20(4), 1987.

Die einzige prüfbare Prognose des Fachs. *Sie ist eng gefasst, gut begründet und hält seit vierzig Jahren. Zwölf Seiten, die man vor jeder Werkzeugentscheidung lesen sollte.*

[7] Conway, M.: *How Do Committees Invent?* Datamation 14(5), 1968.

Sechs Seiten mit einer Beobachtung, die zum Gesetz wurde. Conway schrieb sie beiläufig; ihre Tragweite — dass man Architektur über Organisation steuern kann — wurde erst dreißig Jahre später ausgeschöpft.

[8] DeMarco, T., Lister, T.: *Peopleware*. Dorset House 1987.

Das einzige Buch, das den Menschen als Hauptfaktor ernst nimmt. *Untersucht Arbeitsumgebung, Störungen und Fluktuation — und findet Effekte, die größer sind als alles, was Vorgehensmodelle je gezeigt haben.*

[9] Dijkstra, E.: *Go To Statement Considered Harmful*. CACM 11(3), 1968.

Ein Leserbrief von anderthalb Seiten, der eine jahrzehntelange Auseinandersetzung auslöste. Das Argument ist nicht Ästhetik, sondern Nachvollziehbarkeit: Ohne Sprünge lässt sich der Programmzustand aus der Textstelle ablesen.

[10] Dijkstra, E.: *A Discipline of Programming*. Prentice Hall 1976.

Entwickelt Programme aus ihrer Angabe statt umgekehrt. Anspruchsvoll und für die Praxis wenig unmittelbar brauchbar — aber die Schleifeninvariante als Werkzeug des Verstehens stammt von hier.

[11] Feathers, M.: *Working Effectively with Legacy Code*. Prentice Hall 2004.

Das nützlichste Buch für die tatsächliche Arbeit. *Behandelt den Normalfall, den die Lehrbücher übergehen: ein System ohne Tests, das man ändern muss. Der Begriff der Naht und der Kunstgriff des festhaltenden Tests stammen von hier.*

[12] Forsgren, N., Humble, J., Kim, G.: *Accelerate*. IT Revolution 2018.

Die breiteste vorliegende Erhebung zu wirksamen Praktiken. Methodisch angreifbar — Selbstauskunft, Selbstauswahl, offene Kausalität — und trotzdem die belastbarste Datengrundlage, die das Fach hat. Genau so sollte man sie behandeln.

[13] Fowler, M.: *Refactoring*. 2. Aufl., Addison-Wesley 2018.

Der Katalog der Umgestaltungsschritte, jeweils mit Anlass, Vorgehen und Gegenanzeigen. Der wichtigste Satz steht früh: Ohne Prüfnetz ist es keine Umgestaltung.

[14] Fowler, M.: *Patterns of Enterprise Application Architecture*. Addison-Wesley 2002.

Beschreibt die wiederkehrenden Bauformen betrieblicher Anwendungen und, wichtiger, wann man sie nicht *braucht. Die Abwägungen sind sorgfältiger dargestellt als in den meisten Musterkatalogen.*

[15] Gamma, E., Helm, R., Johnson, R., Vlissides, J.: *Design Patterns*. Addison-Wesley 1994.

Sein größter Beitrag ist das Vokabular, nicht der Katalog. *Ein Wort statt zehn Sätze — bei einem Fach, dessen Engpass die Verständigung ist, ist das viel wert. Als Bauvorschrift missverstanden hat es erheblichen Schaden angerichtet.*

[16] Hoare, C. A. R.: *An Axiomatic Basis for Computer Programming*. CACM 12(10), 1969.

Begründet, wie man über Programme beweisbare Aussagen macht. Aus dem Tripel $\{P\}\ S\ \{Q\}$ sind sowohl die formale Verifikation als auch Meyers Verträge hervorgegangen.

[17] Hoare, C. A. R.: *The Emperor's Old Clothes*. CACM 24(2), 1981.

Die ehrlichste Turing-Award-Rede. *Hoare berichtet über gescheiterte Projekte, an denen er beteiligt war, und über eine Sicherheitsprüfung, die aus Geschwindigkeitsgründen abgeschaltet wurde. Fünfzehn Jahre vor Ariane 5.*

[18] Humble, J., Farley, D.: *Continuous Delivery*. Addison-Wesley 2010.

Beschreibt die durchgängige Strecke von der Übergabe bis zur Auslieferung. Der Kern ist keine Werkzeugfrage, sondern die Forderung, dass der Hauptzweig jederzeit auslieferbar bleibt.

[19] Jones, C.: *Applied Software Measurement*. 3. Aufl., McGraw-Hill 2008.

Jahrzehnte gesammelter Projektdaten. Die Zahlen sind mit Vorsicht zu lesen — Herkunft und Vergleichbarkeit sind teilweise unklar — und es gibt keine breitere Sammlung.

[20] Kernighan, B., Pike, R.: *The Practice of Programming*. Addison-Wesley 1999.

Handwerk statt Methodik: Fehlersuche, Prüfung, Werkzeuge, Portabilität. Zweihundert Seiten ohne ein einziges Vorgehensmodell — und praktisch nützlicher als vieles, was seither erschienen ist.

[21] Lehman, M.: *Programs, Life Cycles, and Laws of Software Evolution*. Proc. IEEE 68(9), 1980.

Die einzige empirisch begründete Gesetzmäßigkeit des Fachs. *Aus der Beobachtung großer Systeme über Jahrzehnte gewonnen. Das Wichtigste steht in einer Einschränkung: Die Komplexität wächst,* sofern nicht gegengearbeitet wird.

[22] Leveson, N.: *Engineering a Safer World*. MIT Press 2011.

Untersucht Unfälle als Systemversagen statt als Bauteilfehler. Zeigt, dass fast alle schweren Softwareunfälle nicht auf falschen Code, sondern auf unvollständige oder widersprüchliche Vorgaben zurückgehen. Frei verfügbar.

[23] McConnell, S.: *Code Complete*. 2. Aufl., Microsoft Press 2004.

Umfassend, praxisnah und mit Belegstellen für jede Empfehlung. Wenn Sie ein einziges Handbuch zur täglichen Arbeit suchen, dieses — und schlagen Sie nach, statt es durchzulesen.

[24] Meyer, B.: *Object-Oriented Software Construction*. 2. Aufl., Prentice Hall 1997.

Design by Contract in voller Ausführung*, zusammen mit einer Analyse des Ariane-Unglücks. Meyer schreibt streitbar und begründet sorgfältig; die Sprache Eiffel ist Nische geblieben, die Ideen sind überall.*

[25] Naur, P., Randell, B. (Hrsg.): *Software Engineering: Report on a Conference*. NATO 1969.

Das Gründungsdokument. *Dokumentiert die Diskussionen wörtlich, samt Widerspruch und Ratlosigkeit. Die Tagesordnung von 1968 liest sich wie das Inhaltsverzeichnis heutiger Lehrbücher — eine ernüchternde Lektüre. Frei verfügbar.*

[26] Ousterhout, J.: *A Philosophy of Software Design*. Yaknyam Press 2018.

Hundertneunzig Seiten gegen die Zergliederung. Fordert tiefe Schnittstellen — wenig nach außen, viel dahinter — und widerspricht ausdrücklich der Regel, Funktionen möglichst klein zu halten. Die Auseinandersetzung mit dieser Gegenposition lohnt.

[27] Parnas, D.: *On the Criteria To Be Used in Decomposing Systems into Modules*. CACM 15(12), 1972.

Fünf Seiten, die den Modulbegriff begründet haben. *Stellt zwei Zerlegungen desselben Programms gegenüber und zeigt, dass die naheliegende die schlechtere ist. Das Kriterium — welche Änderung betrifft wie viele Module — gilt unverändert.*

[28] Royce, W.: *Managing the Development of Large Software Systems*. IEEE WESCON 1970.

Lesen Sie den Text, nicht die zweite Abbildung. *Royce führt das Stufenmodell vor, erklärt, dass es zum Scheitern einlädt, und schlägt Rückkopplungen und einen Prototyp vor. Ein Lehrstück darüber, wie ein Bild einen Text besiegt.*

Wenn Sie nur drei Bücher kaufen

[5]	**Brooks**	Für das Verständnis, warum es schwer ist
[11]	**Feathers**	Für die Arbeit, die tatsächlich anfällt
[23]	**McConnell**	**Zum Nachschlagen im Alltag**

Brooks erklärt den Gegenstand, Feathers den Normalfall, McConnell die Handgriffe. Auffällig ist, was fehlt: kein Buch über ein Vorgehensmodell. Das ist kein Versehen — die Wirkung eines Vorgehensmodells ist die am schwächsten belegte Größe dieses Fachs.

Zwei Ergänzungen: [27] Parnas, fünf Seiten, die man in zwanzig Minuten liest und danach anders zerlegt. Und [25] der NATO-Bericht — weniger als Lehrbuch denn als Mahnung, wie wenig sich seit 1968 an den Fragen geändert hat.